

Documento de propuesta de desarrollo de software I, II y III

Dungeon SCHOOL



INTEGRANTES:

BRAYAN ANDRÉS RAMOS FUENTES

 bramosfuentes42@correo.unicordoba.edu.co 

EMILIO JOSÉ PLAZA VALDÉS

 eplazavaldes@correo.unicordoba.edu.co 

DIEGO ANDRÉS SALAS ALVARÉZ

 dasalasalvarez@correo.unicordoba.edu.co 

MARÍA LUCIA DILSO MOSQUERA

 mdilsomosquera@correo.unicordoba.edu.co 

TUTOR: ALEXANDER TOSCANO RICARDO

https://github.com/area-de-informatica/ds1_pa_dungeonschool

Descripción del videojuego educativo

Se busca crear y diseñar un videojuego original para ordenadores ya sea flash o instalable que permita la evaluación y diseño de estas en diferentes áreas centradas en temas específicos.

Dichas evaluaciones dependiendo el área estarán divididas en 2 sesiones donde en la primera se centrarán en preguntas esparcidas por un mapa que parodia a un salón de clases donde también abran ítems de ayuda de manera aleatoria y la segunda sección se centrará en la representación de un maestro como jefe final donde estará el grueso de la evaluación que se está aplicando de igual manera se podrán usar ítems sobrantes de primera sección en la parte del jefe.

El videojuego tendrá temas y preguntas prefabricadas y de diferentes dificultades de igual manera se buscará en próximas actualizaciones que estas puedan ser más personalizables al igual que los temas centrales o que se implemente la capacidad de integrada por IA de generar preguntas pertinentes en base a un plan de periodo o plan de clases.

ETAPA 1 DISEÑO DE LA APLICACIÓN Y ANÁLISIS DE REQUISITOS 5

1. INTRODUCCIÓN..... 5

PROPÓSITO DEL DOCUMENTO..... 5

ALCANCE DEL PROYECTO MÓDULO DE PIZARRA COMPARTIDA 7

DEFINICIONES Y ACRÓNIMOS 9

2. DESCRIPCIÓN GENERAL 9

OBJETIVOS DEL SISTEMA..... 9

FUNCIONALIDAD GENERAL 9

USUARIOS DEL SISTEMA..... 10

RESTRICCIONES 11

3. REQUISITOS FUNCIONALES 11

CASOS DE USO 11

DIAGRAMAS DE FLUJO DE CASOS DE USO 13

DESCRIPCIÓN DETALLADA DE CADA CASO DE USO; **ERROR! MARCADOR NO DEFINIDO.**

PRIORIDAD DE REQUERIMIENTOS 32

4. REQUISITOS NO FUNCIONALES 33

REQUISITOS DE DESEMPEÑO 33

REQUISITOS DE SEGURIDAD 33

REQUISITOS DE USABILIDAD 33

REQUISITOS DE ESCALABILIDAD 34

5. MODELADO E/R..... 35

DIAGRAMA DE ENTIDAD-RELACIÓN..... 35

DIAGRAMA RELACIONAL 36

SCRIPT DE MODELO RELACIONAL 37

DESCRIPCIÓN DE ENTIDADES Y RELACIONES 38

REGLAS DE INTEGRIDAD REFERENCIAL..... 40

COLECCIONES (NoSLQ) 41

6. ANEXOS..... 42

DIAGRAMAS ADICIONALES 42

REFERENCIAS 42

ETAPA 2: PERSISTENCIA DE DATOS CON BACKEND 43

7. INTRODUCCIÓN 43

PROPÓSITO DE LA ETAPA..... 43

ALCANCE DE LA ETAPA..... 43

DEFINICIONES Y ACRÓNIMOS 43

8. DISEÑO DE LA ARQUITECTURA DE BACKEND 44

DESCRIPCIÓN DE LA ARQUITECTURA PROPUESTA 44

COMPONENTES DEL BACKEND 45

DIAGRAMAS DE ARQUITECTURA 46

9. ELECCIÓN DE LA BASE DE DATOS..... 47

EVALUACIÓN DE OPCIONES (SQL o NoSQL) 48

JUSTIFICACIÓN DE LA ELECCIÓN 49

DISEÑO DE ESQUEMA DE BASE DE DATOS..... 49

10. IMPLEMENTACIÓN DEL BACKEND 51

ELECCIÓN DEL LENGUAJE DE PROGRAMACIÓN	51
CREACIÓN DE LA LÓGICA DE NEGOCIO	51
DESARROLLO DE ENDPOINTS Y APIs	52
AUTENTICACIÓN Y AUTORIZACIÓN	52
11. CONEXIÓN A LA BASE DE DATOS	53
CONFIGURACIÓN DE LA CONEXIÓN	53
DESARROLLO DE OPERACIONES CRUD	54
MANEJO DE TRANSACCIONES.....	54
12. PRUEBAS DEL BACKEND.....	55
DISEÑO DE CASOS DE PRUEBA	55
EJECUCIÓN DE PRUEBAS UNITARIAS Y DE INTEGRACIÓN	56
MANEJO DE ERRORES Y EXCEPCIONES	62
ETAPA 3: CONSUMO DE DATOS Y DESARROLLO FRONTEND	64
13. INTRODUCCIÓN.....	64
PROPÓSITO DE LA ETAPA.....	64
ALCANCE DE LA ETAPA.....	64
DEFINICIONES Y ACRÓNIMOS	64
14. CREACIÓN DE LA INTERFAZ DE USUARIO (UI).....	64
DISEÑO DE LA INTERFAZ DE USUARIO (UI) CON HTML Y CSS	64
CONSIDERACIONES DE USABILIDAD	64
MAQUETACIÓN RESPONSIVA.....	64
15. PROGRAMACIÓN FRONTEND CON JAVASCRIPT (JS)	64
DESARROLLO DE LA LÓGICA DEL FRONTEND	64
MANEJO DE EVENTOS Y COMPORTAMIENTOS DINÁMICOS	64
USO DE BIBLIOTECAS Y FRAMEWORKS (SI APLICABLE).....	64
16. CONSUMO DE DATOS DESDE EL BACKEND.....	64
CONFIGURACIÓN DE CONEXIONES AL BACKEND	64
OBTENCIÓN Y PRESENTACIÓN DE DATOS	64
ACTUALIZACIÓN EN TIEMPO REAL (SI APLICABLE)	65
17. INTERACCIÓN USUARIO-INTERFAZ	65
MANEJO DE FORMULARIOS Y VALIDACIÓN DE DATOS	65
IMPLEMENTACIÓN DE FUNCIONALIDADES INTERACTIVAS	65
MEJORAS EN LA EXPERIENCIA DEL USUARIO	65
18. PRUEBAS Y DEPURACIÓN DEL FRONTEND	65
DISEÑO DE CASOS DE PRUEBA DE FRONTEND	65
PRUEBAS DE USABILIDAD	65
DEPURACIÓN DE ERRORES Y OPTIMIZACIÓN DEL CÓDIGO	65
19. IMPLEMENTACIÓN DE LA LÓGICA DE NEGOCIO EN EL FRONTEND	65
MIGRACIÓN DE LA LÓGICA DE NEGOCIO DESDE EL BACKEND (SI NECESARIO)....	65
VALIDACIÓN DE DATOS Y REGLAS DE NEGOCIO EN EL FRONTEND	65
20. INTEGRACIÓN CON EL BACKEND	65
VERIFICACIÓN DE LA COMUNICACIÓN EFECTIVA CON EL BACKEND	65
PRUEBAS DE INTEGRACIÓN FRONTEND-BACKEND	65
ANEXOS.....	66

Etapas 1 Diseño de la Aplicación y Análisis de Requisitos

1. Introducción

Propósito del Documento

El presente documento tiene como finalidad documentar el proceso de diseño, análisis e implementación del videojuego educativo en su etapa inicial. El desarrollo se basará en la metodología SECMALI, la cual permitirá estructurar y validar cada fase del proyecto para garantizar su efectividad pedagógica y técnica.

El documento se divide en tres etapas para facilitar su entendimiento y aplicación en la asignatura de diseño de software.

Etapas 1 Diseño de la Aplicación y Análisis de Requisitos

Diseño de la Aplicación y Análisis de Requisitos, Esta fase se enfoca en definir los elementos fundamentales del videojuego educativo, considerando su estructura, mecánicas y requerimientos técnicos. El objetivo es establecer una base sólida que permita desarrollar una experiencia lúdica centrada en el aprendizaje y la evaluación de conocimientos en un entorno simulado tipo aula escolar.

El videojuego estará dividido en dos secciones principales:

1. Exploración del mapa:

El jugador recorrerá un entorno en pixel art que representa un aula de clases. A lo largo del mapa encontrará preguntas educativas de dificultad variada, así como ítems de ayuda dispersos aleatoriamente que podrán ser usados más adelante.

2. Enfrentamiento con el maestro (jefe final):

Tras completar la exploración, el jugador se enfrentará al “jefe final”, representado por un maestro. En esta fase se evaluarán los conocimientos adquiridos mediante preguntas más complejas, y se podrán utilizar los ítems recolectados previamente para facilitar el progreso.

Objetivos del diseño:

- Evaluar el aprendizaje de los estudiantes de forma dinámica y entretenida.
- Fomentar el uso de la tecnología como recurso pedagógico.
- Motivar a los usuarios mediante retos progresivos y retroalimentación visual.
- Crear un entorno inmersivo adaptado a la estética pixel art.

Requerimientos funcionales:

- Sistema de preguntas por niveles o temas.
- Registro de respuestas correctas e incorrectas.
- Uso estratégico de ítems recolectados.
- Interfaz intuitiva para navegación y combate.
- Escenarios interactivos con diseño en pixel art.

Requerimientos no funcionales:

- Compatibilidad con ordenadores de bajos recursos.
- Bajo consumo de ancho de banda en caso de conexión al servidor.
- Facilidad de actualización y personalización de preguntas.
- Diseño atractivo y accesible para estudiantes de distintos niveles.

Etapas 2: Persistencia de Datos con Backend – Servidor

En esta etapa se desarrollará el backend que permitirá almacenar y gestionar la información generada durante el uso del videojuego. Esto incluye el progreso de los jugadores, respuestas seleccionadas, estadísticas de juego y configuraciones personalizadas.

Objetivos:

- Crear una API que permita el almacenamiento y recuperación de datos del juego.
- Implementar una base de datos para guardar el progreso del jugador, resultados por nivel, preguntas y configuración de ítems.
- Facilitar la personalización futura del contenido por parte de docentes (subida de preguntas propias, configuraciones de dificultad).

Componentes:

- Endpoints para el registro y login de usuarios.
- Registro de progreso por sesión.
- Almacenamiento de preguntas y resultados por jugador.
- Carga dinámica de datos (preguntas, niveles, ítems).

Tecnologías sugeridas:

- Lenguaje: JavaScript o TypeScript.
- Framework: Express.js o Nest.js.
- Base de datos: MongoDB (NoSQL) o PostgreSQL (SQL), según necesidades.
- API: RESTful con autenticación mediante JWT.
- Herramientas adicionales: Mongoose (si se usa MongoDB), Prisma (si se usa PostgreSQL).

Etapas 3: Consumo de Datos y Desarrollo Frontend – Cliente

Esta etapa contempla la implementación de la interfaz visual y funcional del videojuego en estilo pixel art, centrado en brindar una experiencia lúdica y educativa atractiva.

Objetivos:

- Desarrollar una interfaz gráfica en pixel art que represente un aula y escenarios de batalla con el jefe final.
- Implementar mecánicas de exploración, interacción con preguntas y uso de ítems.
- Conectar el cliente con el backend para cargar preguntas, enviar respuestas y registrar resultados.

Características:

- Sección de exploración estilo RPG 2D con estética pixel art.
- Mecánicas para recolectar ítems y resolver preguntas en un entorno interactivo.
- Sección de jefe final donde se aplican preguntas más complejas.
- Uso de los ítems recolectados como ayudas o potenciadores.
- Feedback visual retro acorde a la estética pixel art (barras de energía, cuadros de texto tipo RPG).

Tecnologías sugeridas:

- Motor/Framework para 2D en pixel art: Phaser.js o Godot (GDScript) si se quiere mantener libre de Unity.
- Lenguaje: JavaScript (Phaser) o GDScript (Godot).
- Recursos visuales: Sprites 2D en pixel art, tilesets para el mapa del aula, animaciones sencillas.
- Sonido: Efectos retro y música chiptune para ambientar.

Alcance del Proyecto Dungeon School

Dungeon School es una propuesta innovadora de videojuego educativo multiplataforma que busca revolucionar el aprendizaje en entornos digitales. Mezclando mecánicas de juegos de rol con elementos pedagógicos, este proyecto se plantea como una herramienta divertida e interactiva para reforzar conocimientos en el aula o desde casa. Ambientado en un universo escolar con estética pixel art retro, el juego permitirá a los estudiantes recorrer mapas tipo aula, resolver retos académicos, recolectar ítems útiles y enfrentarse a un "maestro jefe" para demostrar lo aprendido.

El objetivo principal es motivar a los estudiantes a aprender jugando, brindando una experiencia inmersiva y significativa. Además, el juego está diseñado para expandirse con el tiempo, integrando funciones cada vez más complejas que permitan adaptar el contenido a distintos niveles educativos, personalizar la experiencia del usuario y fomentar el trabajo colaborativo. Este documento detalla el alcance inicial y propone una clasificación de funcionalidades futuras según la complejidad que implicaría su desarrollo e implementación.

Funcionalidades actuales:

- Movimiento del personaje (Arriba, abajo, izquierda, derecha)
- Desbloqueo de zonas
- Interacción con preguntas
- Recolección de ítems
- Uso de ítems
- Enfrentamiento con un enemigo
- Registro de respuestas
- Estado del personaje (barras de energía, barra de estado, etc.)

Funcionalidades futuras:

Funciones Básicas (fácil implementación)

- Guardar el progreso del jugador automáticamente.
- Personalizar el avatar del estudiante con ropa o accesorios.
- Elegir entre distintos mapas escolares para explorar.
- Obtener pistas al ver contenido educativo dentro del juego.
- Subir puntuaciones a un ranking global o escolar.
- Conseguir medallas por logros educativos.
- Cambiar la dificultad del juego según el nivel del estudiante.
- Escuchar narraciones o audios explicativos como ayuda extra.
- Compartir resultados o certificados con padres o docentes.
- Añadir minijuegos entre preguntas para mantener la motivación.
- Personalizar el fondo o estilo del aula virtual.
- Acceder a estadísticas personales de rendimiento.
- Guardar partidas y continuar desde donde se dejó.

Funciones Medias (requieren planificación y desarrollo intermedio)

- Sistema de progresión con logros.
- Versión portable offline.
- Desbloquear niveles temáticos (ciencias, historia, etc.).
- Usar objetos especiales para resolver preguntas más rápido.
- Participar en torneos académicos entre colegios.

- Chatear con otros jugadores dentro del aula virtual.
- Usar animaciones o reacciones del maestro jefe según el rendimiento.
- Integrar videos educativos antes de ciertas preguntas clave.
- Permitir a los profes crear mini misiones con recompensas.
- Recibir retroalimentación automática según errores comunes.
- Jugar con teclado, mouse, pantalla táctil o control.
- Descargar preguntas desde una nube educativa segura.
- Acceder a una biblioteca con guías de estudio integradas.
- Usar un sistema de energía para regular el tiempo de juego.
- Hacer misiones diarias o semanales con objetivos específicos.

Funciones Complejas (altamente personalizadas o tecnológicamente avanzadas)

- Personalización de pruebas por parte del docente.
- Generación automática de preguntas con IA según el plan de clase.
- Modo multijugador (en tiempo real o por turnos).
- Participar en eventos mensuales con recompensas especiales.
- Usar comandos por voz para responder o navegar.
- Desbloquear zonas secretas si se alcanza cierto puntaje.
- Usar monedas virtuales para desbloquear contenido educativo extra.

Definiciones y Acrónimos

- API: Interfaz de Programación de Aplicaciones (Application Programming Interface).
- DBMS: Sistema de Gestión de Bases de Datos (Database Management System).
- JWT: Token Web JSON (JSON Web Token), utilizado para autenticar usuarios.
- CRUD: Crear, Leer, Actualizar y Borrar (Create, Read, Update, Delete).
- Pixel Art: Estilo gráfico basado en píxeles visibles, común en videojuegos retro.
- RPG: Juego de Rol (Role-Playing Game), estilo de juego donde el jugador asume el papel de un personaje en una narrativa.

2. Descripción General

Objetivos del Sistema

El objetivo del sistema es proporcionar una pizarra compartida dentro de un Sistema de Gestión de Contenido llamado CREAVI que permita a los usuarios colaborar de manera eficiente y efectiva, facilitando la creación, visualización y edición de contenido visual en tiempo real. Esta pizarra compartida se diseñará con el propósito de mejorar la comunicación y la colaboración en un entorno en línea, ofreciendo a los usuarios una plataforma intuitiva y versátil para crear y compartir ideas, diagramas, esquemas y contenido visual de manera colaborativa, enriqueciendo así la experiencia de usuario y la productividad en el uso del CMS.

Funcionalidad General

- **Creación y Edición Colaborativa:** Permite a los usuarios crear y editar contenido en la pizarra de forma colaborativa en tiempo real. Múltiples usuarios pueden trabajar en el mismo documento simultáneamente.
- **Herramientas de Dibujo y Anotación:** Proporciona herramientas de dibujo, pinceles, formas y opciones de anotación que permiten a los usuarios plasmar sus ideas y conceptos de manera visual.
- **Carga de Imágenes y Multimedia:** Permite a los usuarios cargar imágenes, videos y otros medios directamente en la pizarra, lo que facilita la ilustración de conceptos.
- **Organización de Contenido:** Ofrece opciones para organizar y estructurar el contenido en la pizarra, como la creación de capas, agrupación de elementos y uso de etiquetas.
- **Historial de Revisiones:** Registra un historial de revisiones que permite a los usuarios rastrear los cambios realizados en la pizarra y restaurar versiones anteriores si es necesario.
- **Compartir y Colaborar:** Permite compartir la pizarra con otros usuarios a través de enlaces o invitaciones, lo que facilita la colaboración con colegas, clientes o amigos.
- **Comentarios y Chat en Tiempo Real:** Los usuarios pueden comentar y discutir sobre el contenido de la pizarra a través de un chat en tiempo real, lo que facilita la comunicación durante la colaboración.
- **Exportación e Impresión:** Ofrece la capacidad de exportar el contenido de la pizarra en varios formatos (PDF, imagen, etc.) y la opción de imprimirlo.
- **Integración con el CMS:** Se integra de manera transparente con el sistema de gestión de contenido (CMS CREAVI), lo que permite incrustar pizarras en los contenidos, metodologías o cualquier otro tipo de componente que permita la pizarra.
- **Personalización y Temas:** Permite a los usuarios personalizar la apariencia de la pizarra y seleccionar temas que se adapten a sus necesidades.
- **Acceso Seguro:** Proporciona medidas de seguridad para garantizar que solo los usuarios autorizados puedan acceder y editar la pizarra.
- **Notificaciones y Actualizaciones en Tiempo Real:** Los usuarios reciben notificaciones sobre cambios en la pizarra y pueden ver actualizaciones en tiempo real mientras otros editan.
- **Acceso Móvil:** Ofrece una experiencia de usuario optimizada en dispositivos móviles, permitiendo el acceso y la colaboración desde smartphones y tabletas.
- **Búsqueda y Filtros:** Facilita la búsqueda de contenido en la pizarra y la aplicación de filtros para organizar y encontrar información específica.
- **Gestión de Usuarios y Permisos:** Permite a los administradores gestionar usuarios y definir permisos de acceso y edición.
- **Informes y Analíticas:** Proporciona información sobre el uso de la pizarra, como quién la ha editado, cuándo se realizaron cambios y estadísticas sobre el contenido(XAPI).

Usuarios del Sistema

Los siguientes usuarios pueden interactuar con la pizarra dependiendo de las funcionalidades.

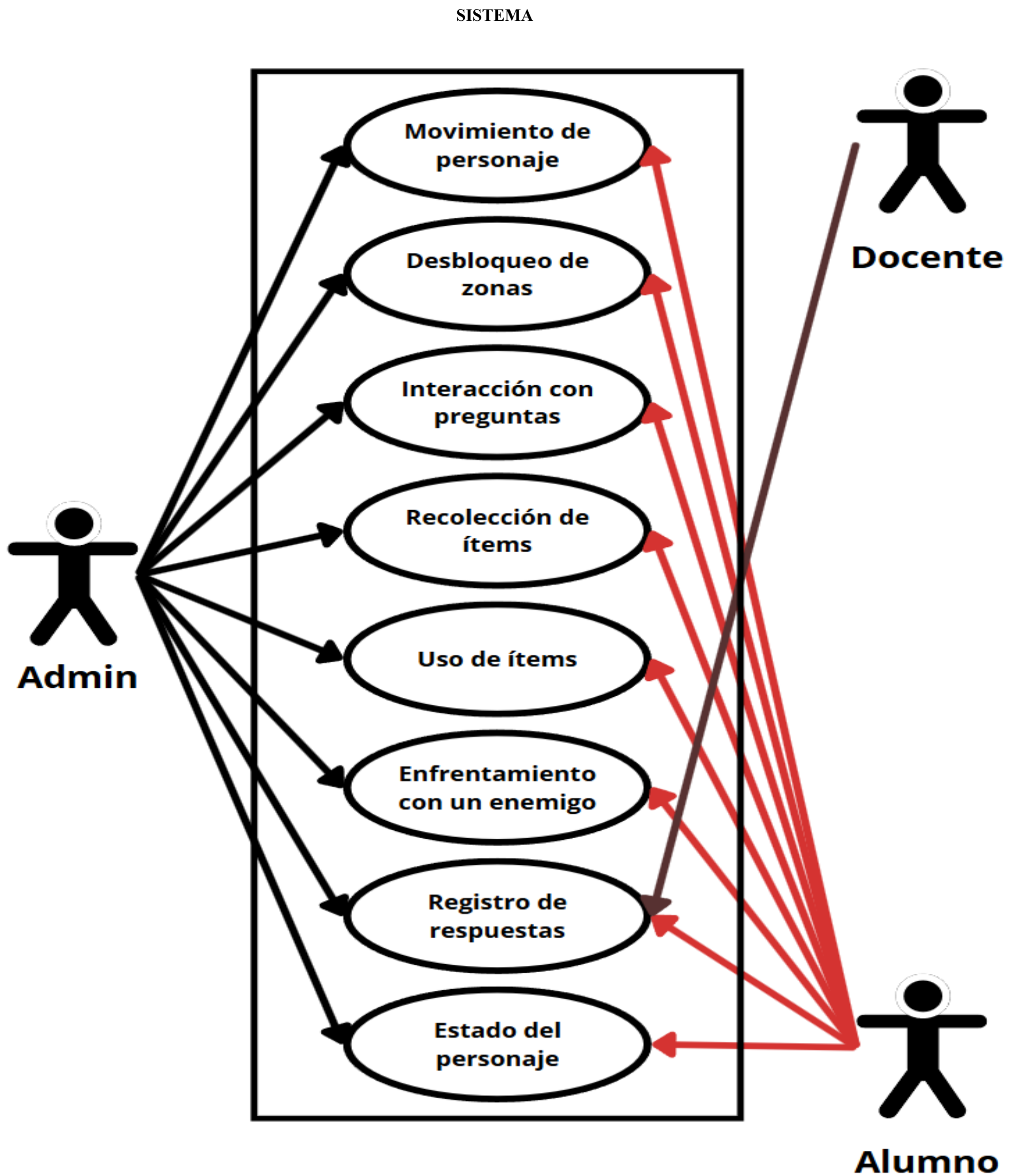
Funcionalidad	Administradores	Docente	Alumno
Movimiento del personaje (Arriba, abajo, izquierda, derecha)	✓		✓
Desbloqueo de zonas	✓		✓
Interacción con preguntas	✓		✓
Recolección de ítems	✓		✓
Uso de ítems	✓		✓
Enfrentamiento con un enemigo	✓		✓
Registro de respuestas	✓	✓	✓
Estado del personaje (barras de energía, barra de estado, etc.)	✓		✓
Guardar el progreso	✓		✓
Personalizar el avatar	✓		✓
Unirse a una partida	✓	✓	✓
Crear nueva partida	✓	✓	
Elegir entre distintos mapas	✓	✓	✓
Obtener pistas	✓		✓
Ranking	✓	✓	✓
Conseguir medallas	✓		✓
Cambiar la dificultad	✓	✓	
Escuchar narraciones	✓		✓
Compartir resultados	✓	✓	✓
Continuar partida	✓	✓	✓
Personalizar el fondo	✓	✓	
Acceder a estadísticas	✓	✓	
Guardar partidas	✓	✓	✓
Crear sesión	✓	✓	✓
Cerrar sesión	✓	✓	✓

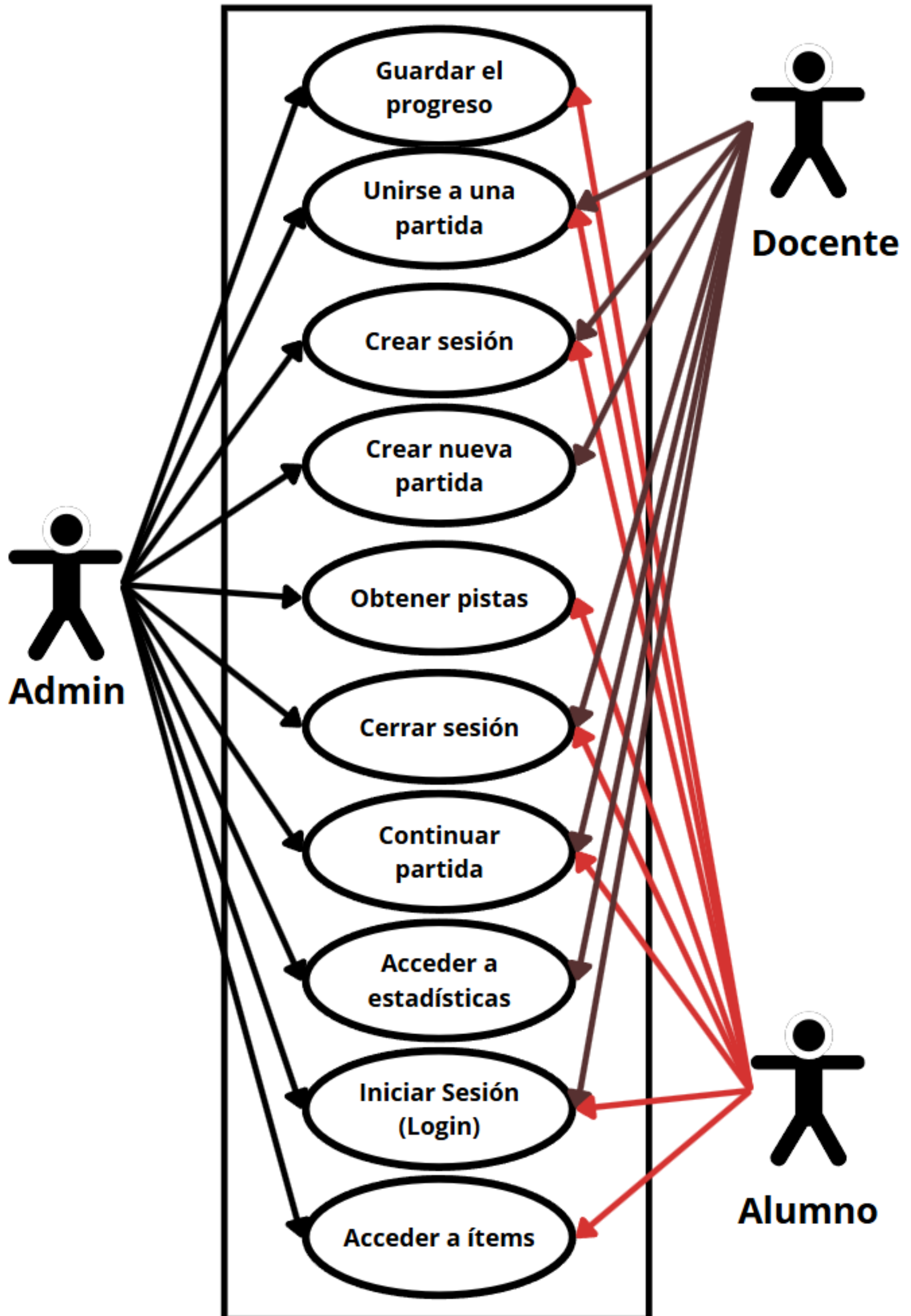
Restricciones

3. Requisitos Funcionales

Casos de Uso

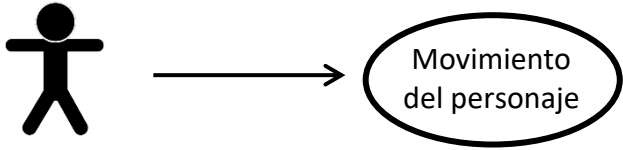
Diagrama de caso de uso





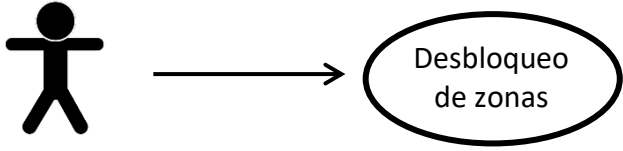
Diagramas de Flujo de Casos de Uso y descripción detallada de cada caso de uso

CASO N°1 Movimiento del personaje

 Alumno - Admin	1. Movimiento del personaje Flujo: Movimiento del personaje Prueba: Desplazarse por el mapa, interactuar con obstáculos y zonas nuevas.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Presionar tecla de dirección AA: Actualizar posición GP: Verificar colisiones	Movimiento del personaje Flujo: SP, AA, GP

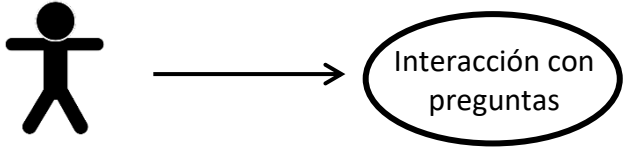
ID	CU-ALU-001	
Nombre	Movimiento del personaje	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno moverse por el entorno del aula virtual mediante las teclas de dirección.	
Urgencia	4	
Esfuerzo	3	
Pre-condiciones	El alumno ha iniciado sesión y se encuentra en el entorno de juego.	
Flujo Normal	Alumno	Sistema
	Presiona una tecla de dirección	
		Detecta la dirección solicitada
		Valida la nueva posición del personaje
		Actualiza la posición del personaje
		Retorna la posición del personaje
		Refleja visualmente el cambio en pantalla
Flujo alternativo 1	Presiona varias teclas a la vez	
		El sistema prioriza una dirección
		Muestra la nueva posición
Flujo alternativo 2	Tecla inválida o sin función	
		Sistema no realiza ninguna acción
Post-condiciones	El personaje se mueve correctamente por el escenario sin errores. El personaje nunca sobrepasa la zona delimitada.	
Excepciones	Error en la carga del escenario o glitch en colisiones.	

CASO N°2 Desbloqueo de zonas

 Alumno - Admin	Desbloqueo de zonas Flujo: Desbloqueo de zonas Prueba: Intentar entrar a zona nueva, cumplir requisitos y acceder.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Interactuar con zona bloqueada AA: Verificar requisitos GP: Habilitar acceso	Desbloqueo de zonas Flujo: SP, AA, GP

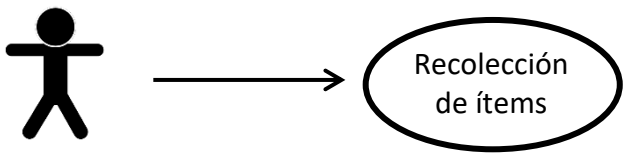
ID	CU-ALU-002	
Nombre	Desbloqueo de zonas	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno acceder a nuevas zonas del mapa al cumplir ciertos requisitos.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El alumno ha recolectado los ítems o respondido las preguntas necesarias.	
Flujo Normal	Alumno	Sistema
	Se ubica en una zona bloqueada	
		Verifica requisitos cumplidos
		Habilita la entrada a la nueva zona
		Reproduce animación de desbloqueo
		Se desbloquea la zona
Flujo alternativo 1	Intenta entrar sin cumplir requisitos	
		Muestra mensaje de acceso denegado
Flujo alternativo 2	Zona desbloqueada previamente	
		Acceso inmediato sin validación
Post-condiciones	El alumno accede correctamente a una nueva sección del juego.	
Excepciones	Bug de colisión impide entrada o error en validación de requisitos.	

CASO N°3 Interacción con preguntas

 Alumno - Admin	Interacción con preguntas Flujo: Interacción con preguntas Prueba: El alumno se acerca a la pregunta, selecciona una opción.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Seleccionar pregunta AA: Elegir respuesta GP: Evaluar y registrar respuesta	Interacción con preguntas Flujo: SP, AA, GP

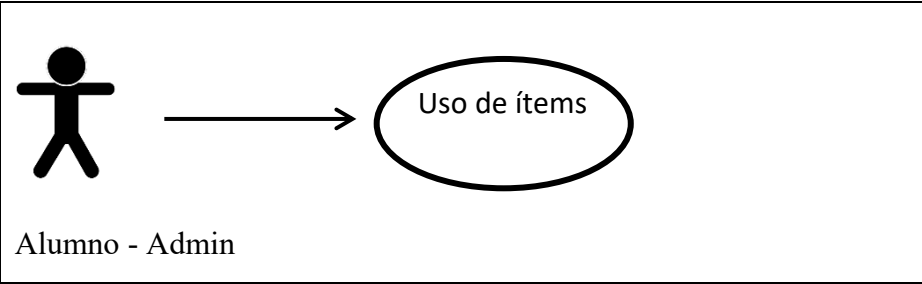
ID	CU-ALU-003	
Nombre	Interacción con preguntas	
Actores	Alumno, Sistema	
Objetivo	Interactuar con preguntas permite al alumno seleccionar y responder las preguntas durante la exploración del juego.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El alumno se encuentra frente a la pregunta visible en el mapa.	
Flujo Normal	Alumno	Sistema
	Selecciona una pregunta	
		Muestra la interfaz de respuestas
	Elige una respuesta	
		Bloquea seleccionador de respuesta
		Evalúa si es correcta o incorrecta
		Registra la respuesta en la base de datos
Flujo alternativo 1	No selecciona ninguna opción	
		Bloquea el botón de “Enviar”
Flujo alternativo 2	Responde incorrectamente	
		Registra la respuesta en la base de datos
Post-condiciones	La respuesta queda registrada y el sistema guarda el resultado.	
Excepciones	Error de conexión con la base de datos o pregunta mal cargada.	

CASO N°4 Recolección de ítems

 Alumno - Admin	Recolección de ítems Flujo: Recolección de ítems Prueba: Tomar ítem del mapa y comprobar que aparece en inventario.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Acercarse a un ítem AA: Recoger ítem GP: Añadir al inventario	Recolección de ítems Flujo: SP, AA, GP

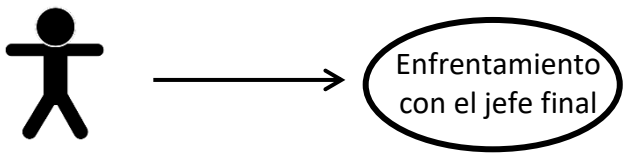
ID	CU-ALU-004	
Nombre	Recolección de ítems	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno recoger ítems del entorno para su posterior uso.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El ítem está visible en el entorno y el jugador puede recolectarlo.	
Flujo Normal	Alumno	Sistema
	Se acerca al ítem	
		Detecta la proximidad
	Presiona la tecla de acción	
		El ítem desaparece del entorno
		Añade el ítem al inventario
		Muestra notificación de recolección
Flujo alternativo 1	El inventario está lleno	
		Muestra mensaje de espacio insuficiente
Flujo alternativo 2	El ítem ya fue recogido antes	
		No ocurre ninguna acción
Post-condiciones	El ítem queda guardado en el inventario del jugador.	
Excepciones	Error en la animación de recogida o ítem duplicado por bug.	

CASO N°5 Uso de ítems

 Alumno - Admin	Uso de ítems Flujo: Uso de ítems Prueba: Usar un ítem desde el inventario y ver su efecto reflejado.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Abrir inventario AA: Seleccionar ítem GP: Aplicar efecto y actualizar	Uso de ítems Flujo: SP, AA, GP

ID	CU-ALU-005	
Nombre	Uso de ítems	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno utilizar ítems recolectados para obtener ventajas en el juego.	
Urgencia	4	
Esfuerzo	3	
Pre-condiciones	El alumno debe tener al menos un ítem en el inventario.	
Flujo Normal	Alumno	Sistema
	Abre el inventario	
		Muestra lista de ítems disponibles
	Selecciona un ítem	
		Se verifica el tipo de ítem
	Confirma el uso del ítem	
		Aplica efecto correspondiente
		Actualiza estado del personaje
		Muestra mensaje de confirmación
Flujo alternativo 1	El ítem no es aplicable en ese momento	
		Muestra advertencia “No se puede usar aquí”
Flujo alternativo 2	El ítem ya fue usado o está agotado	
		Muestra mensaje “Ítem no disponible”
Post-condiciones	El efecto del ítem se aplica correctamente y se actualiza el inventario.	
Excepciones	Error en la base de datos o bug que impide el uso correcto del ítem.	

CASO N° 6 Enfrentamiento con el jefe final

 Alumno - Admin	Enfrentamiento con un enemigo Flujo: Enfrentamiento con un enemigo Prueba: Completar preguntas del jefe final usando ítems recolectados.
Prioridad: Requerido Versión: 1 Complejidad: Alta SP: Iniciar enfrentamiento AA: Contestar preguntas GP: Evaluar desempeño y actualizar estado	Enfrentamiento con un enemigo Flujo: SP, AA, GP

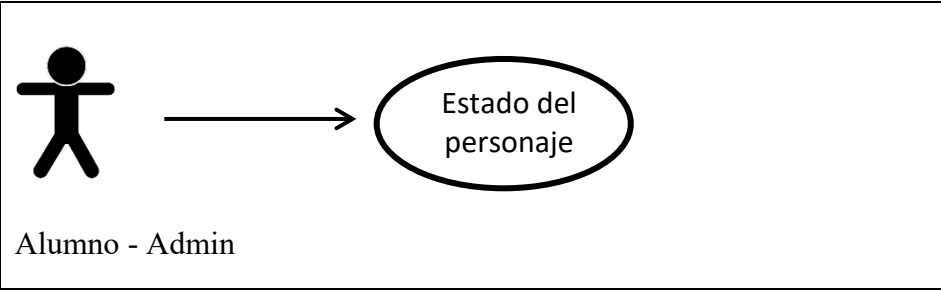
ID	CU-ALU-006	
Nombre	Enfrentamiento con el jefe final	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno enfrentarse al jefe final respondiendo preguntas y utilizando ítems.	
Urgencia	4	
Esfuerzo	2	
Pre-condiciones	El alumno ha completado la fase de exploración y ha recolectado ítems.	
Flujo Normal	Alumno	Sistema
	Inicia enfrentamiento	
		Carga animación del enemigo
	Responde preguntas complejas	
		Evalúa respuestas en tiempo real
	Usa ítems como ayuda	
		Aplica efectos si son válidos
		Muestra retroalimentación visual
		Determina resultado del enfrentamiento
Flujo alternativo 1	Responde incorrectamente varias preguntas	
		Muestra barra de energía reducida
Flujo alternativo 2	Agota sus ítems y continúa	
		Se evalúa solo con conocimientos
Post-condiciones	Se actualiza el estado del juego según el resultado (victoria o derrota).	
Excepciones	Errores de carga de preguntas o ítems no funcionales durante el combate.	

CASO N°7 Registro de respuestas

	Registro de respuestas Flujo: Registro de respuestas Prueba: Responder y verificar que la base de datos almacene el resultado.
Alumno - Docente - Admin	Registro de respuestas Flujo: SP, AA, GP
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Responder pregunta AA: Registrar respuesta GP: Asociar con usuario	

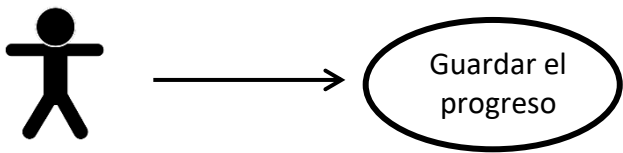
ID	CU-ALU-007	
Nombre	Registro de respuestas	
Actores	Alumno, Sistema	
Objetivo	Permitir al sistema almacenar las respuestas seleccionadas por los alumnos para su posterior análisis.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El alumno ha respondido al menos una pregunta durante el juego.	
Flujo Normal	Alumno	Sistema
	Envía repuesta seleccionada	
		Recibe respuesta seleccionada del alumno
		Registra respuesta en la base de datos
		Retorna mensaje de respuesta guardada
Flujo alternativo 1	El alumno cierra el juego antes de responder	
		La respuesta no se registra
Flujo alternativo 2	La conexión se pierde durante el envío	
		Se reintenta guardar automáticamente
Post-condiciones	La respuesta queda almacenada correctamente y disponible para ser calificadas.	
Excepciones		

CASO N°8 Estado del personaje

 Alumno - Admin	Estado del personaje Flujo: Estado del personaje Prueba: Recibir daño o beneficio y observar cambios en barras visuales.
Prioridad: Requerido Versión: 1 Complejidad: Baja SP: Cambios en juego AA: Actualizar barra de estado GP: Mostrar en pantalla	Estado del personaje Flujo: SP, AA, GP

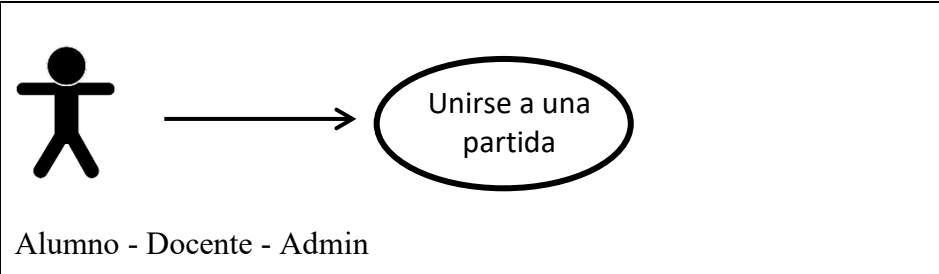
ID	CU-ALU-008	
Nombre	Estado del personaje	
Actores	Alumno, Sistema	
Objetivo	Visualizar en pantalla el estado actual del personaje (energía, barra de progreso, etc.).	
Urgencia	3	
Esfuerzo	1	
Pre-condiciones	El alumno debe estar en una partida activa.	
Flujo Normal	Alumno	Sistema
	Interactúa con el entorno	
		Detecta cambios en el estado (daño, progreso)
		Almacenar en el sistema
		Retornar actualización del estado
		Actualiza visualmente las barras y atributos
		Refleja cambios en tiempo real en pantalla
Flujo alternativo 1	Completa una tarea y sube de nivel	
		Aumenta energía y actualiza barra de estado
Flujo alternativo 2	Nota que la barra no refleja los cambios recientes	
		Detecta error de sincronización
		Recupera el estado correcto
		Actualiza visualmente la barra de estado
Post-condiciones	El estado del personaje queda sincronizado con su estado actual.	
Excepciones	No se actualizan las barras por error en sprites o animaciones.	

CASO N°9 Guardar el progreso

 Alumno - Admin	Guardar el progreso Flujo: Guardar el progreso Prueba: Llegar a zona de guardado o realizar guardado automático y verificar en la siguiente sesión.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Detectar punto de guardado AA: Confirmar guardado GP: Almacenar estado del juego	Guardar el progreso Flujo: SP, AA, GP

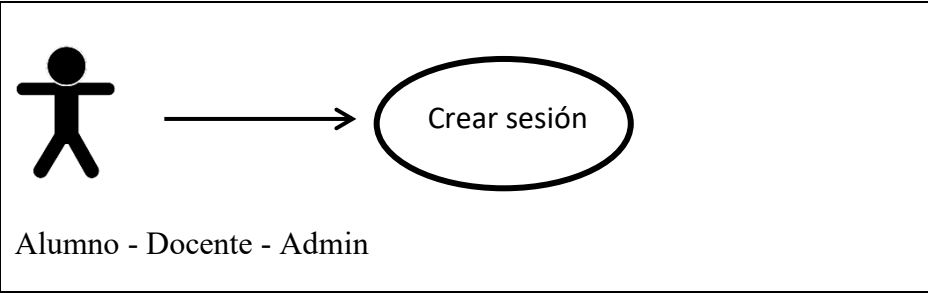
ID	CU-ALU-009	
Nombre	Guardar el progreso	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno guardar su progreso actual en el juego.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El alumno debe estar en una sesión activa del juego.	
Flujo Normal	Alumno	Sistema
	Llega a un punto de guardado o abre el menú	
		Muestra opción de guardar
	Selecciona “Guardar progreso”	
		Detecta y recopila datos actuales
	Confirma guardado	
		Almacena datos en la base de datos
		Muestra mensaje de confirmación
Flujo alternativo 1	Cierra el juego sin guardar manualmente	
		Se activa guardado automático
Flujo alternativo 2	Conexión débil al guardar	
		Se reintenta guardar en segundo plano
Post-condiciones	El progreso del alumno queda guardado correctamente para futuras sesiones.	
Excepciones	Fallo en la conexión o error en la base de datos durante el guardado.	

CASO N° 10 Unirse a una partida

 Alumno - Docente - Admin	Unirse a una partida Flujo: Unirse a una partida Prueba: Elegir una partida activa y validar ingreso exitoso.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Ver partidas disponibles AA: Seleccionar partida GP: Unirse a la sesión	Unirse a una partida Flujo: SP, AA, GP

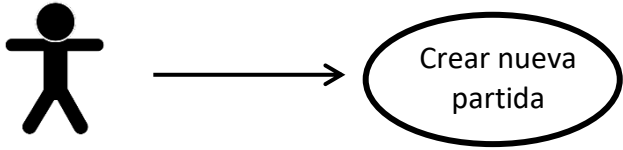
ID	CU-ALDOC-001	
Nombre	Unirse a una partida	
Actores	Alumno, Docente, Sistema	
Objetivo	Permitir a los usuarios unirse a una partida activa previamente creada.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	Debe haber una partida disponible para unirse.	
Flujo Normal	Alumno/Docente	Sistema
	Accede al menú de partidas	
		Muestra lista de partidas disponibles
	Selecciona una partida	
		Verifica disponibilidad
	Confirma unirse	
		Asocia usuario a la partida
		Carga escenario de la partida
Flujo alternativo 1	La partida está llena	
		Muestra mensaje “Partida no disponible”
Flujo alternativo 2	Partida no encontrada o eliminada	
		Muestra advertencia y actualiza lista
Post-condiciones	El usuario se une correctamente a la partida seleccionada.	
Excepciones	Error de sincronización o fallo en carga del entorno.	

CASO N° 11 Crear sesión

 Alumno - Docente - Admin	Crear sesión Flujo: Crear sesión Prueba: Ingresar usuario/contraseña válidos y acceder al juego. Crear sesión Flujo: SP, AA, GP
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Ingresar datos AA: Validar información GP: Generar sesión activa	

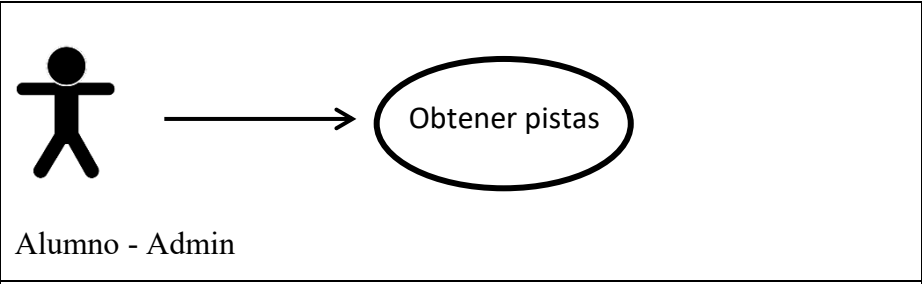
ID	CU-ALDOC-002	
Nombre	Crear sesión	
Actores	Alumno, Docente, Sistema	
Objetivo	Permitir a los usuarios iniciar sesión en el sistema para acceder a sus partidas y funciones.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El usuario debe tener una cuenta registrada.	
Flujo Normal	Alumno/Docente	Sistema
	Abre el menú de inicio de sesión	
		Muestra formulario de login
	Ingresar usuario y contraseña	
		Verifica credenciales
		Crea sesión activa
		Redirige al menú principal
Flujo alternativo 1	Olvida la contraseña	
		Opción de recuperación disponible
Flujo alternativo 2	Credenciales incorrectas	
		Muestra mensaje de error y solicita nuevo intento
Post-condiciones	El usuario inicia sesión y accede a su perfil y progreso.	
Excepciones	Error en servidor de autenticación o token inválido.	

CASO N°12 Crear nueva partida

	Crear nueva partida Flujo: Crear nueva partida Prueba: Configurar una nueva partida y verificar que inicie correctamente.
Docente - Admin	
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Definir parámetros de juego AA: Confirmar creación GP: Iniciar nueva partida	Crear nueva partida Flujo: SP, AA, GP

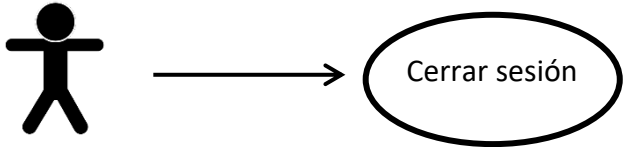
ID	CU-DOC-002	
Nombre	Crear nueva partida	
Actores	Docente, Sistema	
Objetivo	Permitir al docente crear una nueva partida para que los estudiantes puedan unirse.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El docente ha iniciado sesión correctamente.	
Flujo Normal	Docente	Sistema
	Accede al menú de creación de partida	
		Muestra formulario de configuración
	Configura parámetros (nombre, dificultad, etc.)	
		Verifica validez de datos
	Confirma creación	
		Registra la partida en la base de datos
		Muestra mensaje de partida creada con éxito
Flujo alternativo 1	Intenta crear una partida con nombre duplicado	
		Muestra advertencia y solicita cambio
Flujo alternativo 2	No completa los campos requeridos	
		Muestra mensaje de error y bloquea creación
Post-condiciones	La nueva partida queda disponible para ser unida por los estudiantes.	
Excepciones	Fallo en el servidor o error al registrar datos de la partida.	

CASO N°13 Obtener pistas

 Alumno - Admin	Obtener pistas Flujo: Obtener pistas Prueba: Solicitar ayuda en una pregunta y visualizar la pista correspondiente.
Prioridad: Requerido Versión: 1 Complejidad: Baja SP: Activar contenido educativo AA: Esperar reproducción o interacción GP: Mostrar pista al jugador	Obtener pistas Flujo: SP, AA, GP

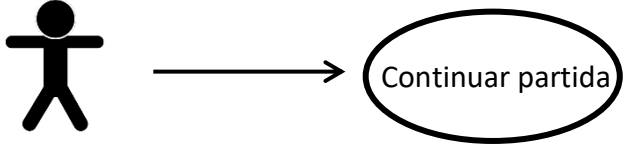
ID	CU-ALU-010	
Nombre	Obtener pistas	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno visualizar pistas para responder preguntas o avanzar en el juego.	
Urgencia	5	
Esfuerzo	1	
Pre-condiciones	El alumno debe estar interactuando con una pregunta o desafío.	
Flujo Normal	Alumno	Sistema
	Solicita ayuda o pista	
		Verifica si hay pista disponible
		Muestra contenido de pista (texto, audio o imagen)
		Permite volver a la pregunta
Flujo alternativo 1	No hay pista disponible para esa pregunta	
		Muestra mensaje: “No disponible”
Flujo alternativo 2	El alumno ya usó la pista	
		Muestra la pista nuevamente como referencia
Post-condiciones	El alumno recibe una pista válida que lo guía en su respuesta.	
Excepciones	Fallo en la carga del contenido multimedia o conexión interrumpida.	

CASO N°14 Cerrar sesión

 Alumno - Docente - Admin	Cerrar sesión Flujo: Cerrar sesión Prueba: Presionar "Cerrar sesión" y verificar redirección al inicio.
Prioridad: Requerido Versión: 1 Complejidad: Baja SP: Seleccionar opción de cerrar sesión AA: Confirmar salida GP: Finalizar sesión y volver al menú	Cerrar sesión Flujo: SP, AA, GP

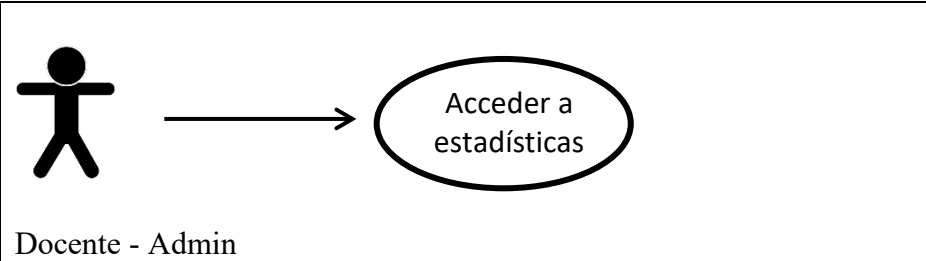
ID	CU-ALDOC-003	
Nombre	Cerrar sesión	
Actores	Alumno, Docente, Sistema	
Objetivo	Permitir al usuario salir del sistema de forma segura.	
Urgencia	5	
Esfuerzo	1	
Pre-condiciones	El usuario debe tener una sesión activa.	
Flujo Normal	Alumno/Docente	Sistema
	Accede al menú de usuario	
		Muestra opciones de cuenta
	Selecciona “Cerrar sesión”	
		Cierra sesión activa
		Redirige al menú de inicio de sesión
Flujo alternativo 1	Intenta cerrar sesión sin conexión	
		Cierra sesión localmente y sincroniza al reconectar
Flujo alternativo 2	No confirma la acción	
		Cancela el proceso y permanece en sesión
Post-condiciones	La sesión se cierra y los datos quedan protegidos.	
Excepciones	Fallo en el cierre de sesión por error de servidor o token inválido.	

CASO N°15 Continuar partida

 Alumno - Docente - Admin	Continuar partida Flujo: Continuar partida Prueba: Iniciar desde último punto guardado y validar datos cargados.
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Acceder a menú de partidas guardadas AA: Seleccionar partida GP: Cargar progreso	Continuar partida Flujo: SP, AA, GP

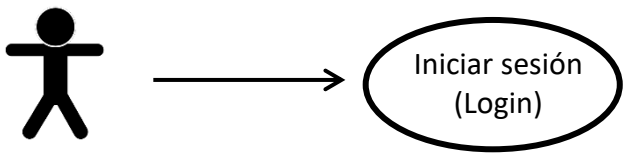
ID	CU-ALDOC-004	
Nombre	Continuar partida	
Actores	Alumno, Docente, Sistema	
Objetivo	Permitir a los usuarios reanudar una partida previamente guardada.	
Urgencia	4	
Esfuerzo	3	
Pre-condiciones	Debe existir una partida guardada asociada al usuario.	
Flujo Normal	Alumno/Docente	Sistema
	Accede al menú de partidas guardadas	
		Muestra lista de partidas anteriores
	Selecciona una partida	
		Recupera datos de guardado
	Confirma reanudar	
		Carga estado y entorno anterior
Flujo alternativo 1	No hay partidas guardadas	
		Muestra mensaje: “No hay partidas disponibles”
Flujo alternativo 2	La partida seleccionada está corrupta	
		Muestra error y ofrece volver al menú
Post-condiciones	El usuario continúa desde el último punto guardado.	
Excepciones	Error de sincronización o fallo en la carga del estado del juego.	

CASO N°16 Acceder a estadísticas

	Acceder a estadísticas Flujo: Acceder a estadísticas Prueba: Iniciar sesión como docente y ver gráficas de rendimiento.
Docente - Admin Prioridad: Requerido Versión: 1 Complejidad: Media SP: Iniciar sesión AA: Navegar al panel de estadísticas GP: Visualizar datos	Acceder a estadísticas Flujo: SP, AA, GP

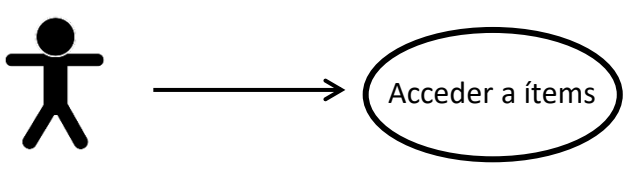
ID	CU-DOC-003	
Nombre	Acceder a estadísticas	
Actores	Docente, Sistema	
Objetivo	Permitir al docente visualizar estadísticas de desempeño de los alumnos.	
Urgencia	5	
Esfuerzo	3	
Pre-condiciones	El docente ha iniciado sesión con permisos válidos.	
Flujo Normal	Docente	Sistema
	Accede al panel de estadísticas	
		Solicita y recupera datos de alumnos
	Selecciona grupo o alumno específico	
		Muestra gráficos, porcentajes y respuestas
		Permite exportar o analizar
Flujo alternativo 1	Filtra por alumno inexistente	
		Muestra mensaje: “No se encontraron datos”
Flujo alternativo 2	Filtra por periodo sin actividad	
		Muestra advertencia y sugiere otro filtro
Post-condiciones	El docente puede tomar decisiones pedagógicas basadas en los datos.	
Excepciones	Error de conexión con la base de datos o estadísticas incompletas.	

CASO N°17 Iniciar Sesión (Login)

	Iniciar sesión (Login) Flujo: Iniciar sesión (Login) Prueba: Ingresar credenciales válidas y acceder al sistema.
Docente - Alumno - Admin Prioridad: Requerido Versión: 1 Complejidad: Media SP: Ingresar datos de usuario (email y contraseña) AA: Validar credenciales contra la base de datos GP: Crear sesión activa y redirigir al menú principal según el rol	Iniciar sesión (Login) Flujo: SP, AA, GP

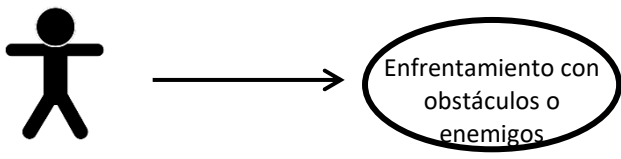
ID	CU-ALUDOADM-001	
Nombre	Iniciar sesión	
Actores	Admin, Docente, Alumno, Sistema	
Objetivo	Permitir a los usuarios iniciar sesión en el sistema para acceder a sus funcionalidades personalizadas.	
Urgencia	5	
Esfuerzo	5	
Pre-condiciones	El usuario debe tener una cuenta previamente registrada en el sistema. El sistema debe estar en línea y operativo.	
Flujo Normal	Alumno	Sistema
	Accede a la pantalla de inicio de sesión	
		Muestra el formulario de login (usuario y contraseña)
	Ingresa usuario y contraseña válidos	
		Verifica las credenciales en la base de datos
		Autentica al usuario y genera token de sesión
		Redirige al usuario al menú principal correspondiente según su rol
Flujo alternativo 1		Muestra mensaje: “Credenciales inválidas. Intente nuevamente”
Flujo alternativo 2		Muestra mensaje: “Usuario no registrado. ¿Desea crear una cuenta?”
Flujo alternativo 3		Muestra mensaje: “Error en el servidor. Intente más tarde”
Post-condiciones	El usuario inicia sesión y accede a sus funcionalidades según su tipo de rol.	
Excepciones	Token inválido Interrupción de la conexión con el backend Ataques por fuerza bruta (protegido mediante bloqueo temporal)	

CASO N°18 Acceder a ítems

 Alumno	Acceder a ítems Flujo: Acceder a ítems Prueba: Abrir el inventario y verificar que se despliegue la lista de ítems recolectados con sus detalles.
Prioridad: Requerido Versión: 1 Complejidad: Baja SP: Abrir inventario AA: Seleccionar ítem para inspeccionar GP: Mostrar detalles del ítem y sus posibles usos.	Acceder a ítems Flujo: SP, AA, GP

ID	CU-ALU-011	
Nombre	Acceder a ítems	
Actores	Alumno, Sistema	
Objetivo	Permitir al alumno ingresar al inventario para visualizar los ítems recolectados, consultar su nombre, tipo, descripción y cantidad disponible, sin generar ningún efecto en el juego.	
Urgencia	4	
Esfuerzo	2	
Pre-condiciones	El alumno debe haber recolectado al menos un ítem durante la exploración del juego.	
Flujo Normal	Alumno	Sistema
	Abre el menú de inventario	
		Muestra la lista de ítems disponibles
	Navega por la lista de ítems	
	Selecciona un ítem	
		Muestra detalles del ítem (nombre, descripción, tipo, cantidad)
	(No realiza ninguna acción adicional)	
		Espera interacción futura (como usar)
Flujo alternativo 1	El alumno no ha recolectado ítems	
		El sistema muestra el mensaje: “Inventario vacío”
Flujo alternativo 2	El alumno selecciona un ítem inválido o dañado	
		El sistema muestra advertencia: “Ítem no disponible”
Flujo alternativo 3	El sistema tarda en cargar los datos	
		Muestra pantalla de carga o reintento
Post-condiciones	El alumno accede correctamente a la información de los ítems y puede prepararse para usarlos.	
Excepciones	Fallos en la base de datos o error en la visualización del inventario.	

Caso N°19 Enfrentamiento con obstáculos o enemigos

 Alumno	Enfrentamiento con obstáculos o enemigos Flujo: Enfrentamiento con obstáculos o enemigos Prueba: Colisionar con un enemigo u obstáculo en el mapa y verificar que se active el evento correspondiente (daño o bloqueo de paso). Enfrentamiento con obstáculos o enemigos Flujo: SP, AA, GP
Prioridad: Requerido Versión: 1 Complejidad: Media SP: Detectar colisión con enemigo u obstáculo AA: Activar secuencia de enfrentamiento o penalización GP: Aplicar efecto al personaje y actualizar estado	

ID	CU-ALU-012	
Nombre	Enfrentamiento con obstáculos o enemigos	
Actores	Alumno, Sistema	
Objetivo	Permitir que el alumno interactúe con enemigos u obstáculos en el entorno, desencadenando una penalización o secuencia de desafío..	
Urgencia	4	
Esfuerzo	3	
Pre-condiciones	El alumno debe estar en una sesión activa del juego y haberse desplazado por el mapa hasta un obstáculo o enemigo.	
Flujo Normal	Alumno	Sistema
	Se desplaza por el mapa	
	se encuentra con un obstáculo o enemigo	
		Detecta la colisión con el objeto o enemigo
		Inicia secuencia de enfrentamiento o aplica penalización (daño, retroceso, etc.)
		Actualiza el estado del personaje en tiempo real
Flujo alternativo 1	Realiza movimiento evasivo a tiempo	
		Cancela daño y mantiene estado del personaje
Flujo alternativo 2	Se acerca a un enemigo visible	
		No detecta interacción y no inicia el enfrentamiento
Post-condiciones	El sistema actualiza correctamente el estado del personaje tras el enfrentamiento o penalización.	
Excepciones	Error en detección de colisión o glitch que permite evitar efectos del obstáculo/enemigo.	

Prioridad de Requerimientos

	Urgencia					
Impacto		1-Baja	2-Menor	3-Moderada	4-Alta	5-Obligatoria
	5-Muy alto					
					CU-11 CU-12	CU-17
	4-Alto					
				CU-18 CU-16	CU-19	
	3-Medio					
			CU-5	CU-15	CU-1	CU-2 CU-3
	2-Bajo					
			CU-7	CU-10	CU-4 CU-6	CU-9
	1-Muy bajo					
				CU-8	CU-13 CU-14	

4. Requisitos No Funcionales

Compatibilidad multiplataforma: El videojuego debe ejecutarse de manera fluida en ordenadores de diferentes gamas, priorizando el bajo consumo de recursos para garantizar su uso en contextos escolares con equipos limitados.

Facilidad de mantenimiento y actualización: El sistema debe permitir incorporar nuevos niveles, preguntas, ítems o personajes sin necesidad de reestructurar el código base.

Estética atractiva: El juego debe tener un diseño visual coherente, centrado en el estilo pixel art, que sea accesible y amigable para estudiantes de diversos niveles.

Modularidad del contenido: Las preguntas y los ítems deben poder cargarse de forma dinámica desde una base de datos para permitir personalización por parte de docentes.

Conectividad opcional: El juego debe ser funcional tanto en modalidad en línea como offline (para versiones futuras portables).

Requisitos de Desempeño

Rendimiento en tiempo real: Las acciones del jugador, como movimiento, uso de ítems o interacción con preguntas, deben reflejarse inmediatamente sin demoras perceptibles.

Carga rápida del juego y sus componentes: Los escenarios, sprites, datos del jugador e ítems deben cargarse de forma eficiente para evitar tiempos de espera innecesarios.

Bajo consumo de CPU/GPU: El juego debe optimizar el uso de recursos para evitar ralentizaciones en máquinas escolares o con hardware limitado.

Resistencia a fallos: El juego debe manejar con gracia errores como caídas de conexión o datos incompletos, permitiendo reintentos o notificaciones al usuario.

Requisitos de Seguridad

Autenticación de usuarios: El sistema debe implementar una capa de autenticación segura para permitir el acceso personalizado de alumnos, docentes y administradores.

Protección de datos personales: Las credenciales, estadísticas y resultados de los jugadores deben estar cifrados y protegidos tanto en tránsito como en almacenamiento.

Gestión de sesiones: Las sesiones deben ser válidas por un tiempo definido y cerrarse automáticamente en caso de inactividad prolongada o múltiples accesos concurrentes.

Registros de actividad: Toda interacción clave del jugador (respuestas, avances, ítems recolectados) debe quedar registrada con fines de análisis pedagógico o recuperación de progreso.

Control de acceso por rol: Las funcionalidades disponibles deben variar según el rol (alumno, docente, admin), con restricciones para evitar cambios no autorizados.

Requisitos de Usabilidad

Interfaz intuitiva: El sistema debe contar con menús y controles simples, accesibles para niños y adolescentes, sin necesidad de formación técnica previa.

Diseño accesible: Los elementos gráficos, colores y textos deben cumplir criterios básicos de accesibilidad visual y cognitiva.

Retroalimentación inmediata: Toda acción del usuario (respuesta, movimiento, selección de ítem) debe generar una respuesta visual clara (sonido, animación, texto).

Compatibilidad con dispositivos de entrada variados: Debe poder jugarse con teclado, mouse o pantalla táctil.

Documentación básica: Se deben ofrecer ayudas visuales, tutoriales iniciales o indicaciones contextuales que orienten al jugador en sus primeras interacciones.

Requisitos de Escalabilidad

Carga de múltiples usuarios: El backend debe permitir que varios estudiantes accedan simultáneamente sin afectar el rendimiento (especialmente en torneos escolares o actividades grupales).

Estructura modular: El juego debe permitir añadir nuevos mapas, preguntas, ítems o mecánicas con facilidad para que evolucione a versiones más completas.

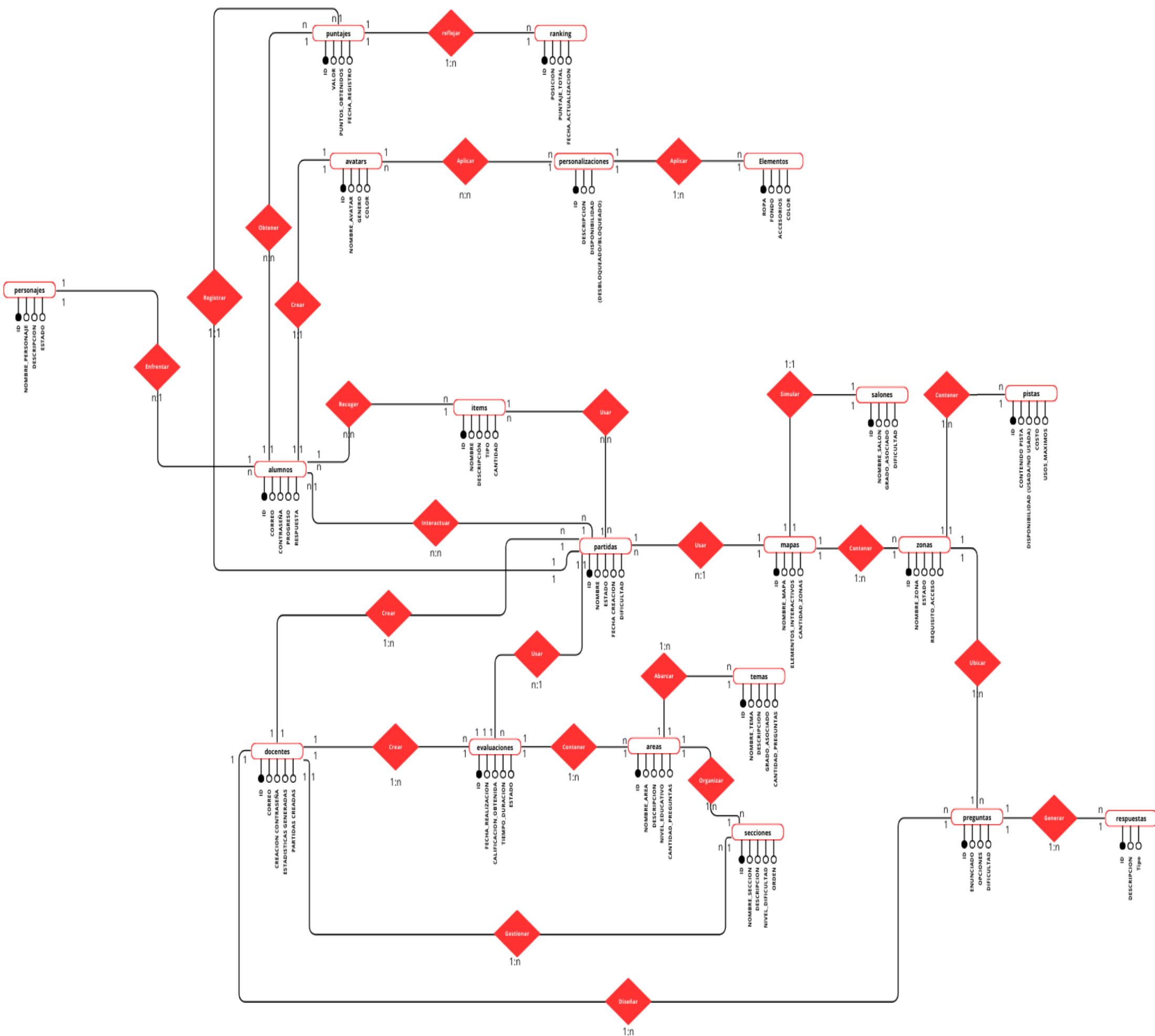
Escalabilidad del servidor: En su versión en línea, el backend debe poder escalar horizontalmente (por ejemplo, usando balanceadores de carga o servicios en la nube).

Soporte para contenido personalizado: En futuras versiones, se debe contemplar la creación de misiones o preguntas por parte de los docentes, escalando no solo en usuarios sino en tipos de contenido.

Integración con servicios externos: Se debe prever la futura conexión con plataformas educativas, analíticas o servicios de IA para generación dinámica de contenido.

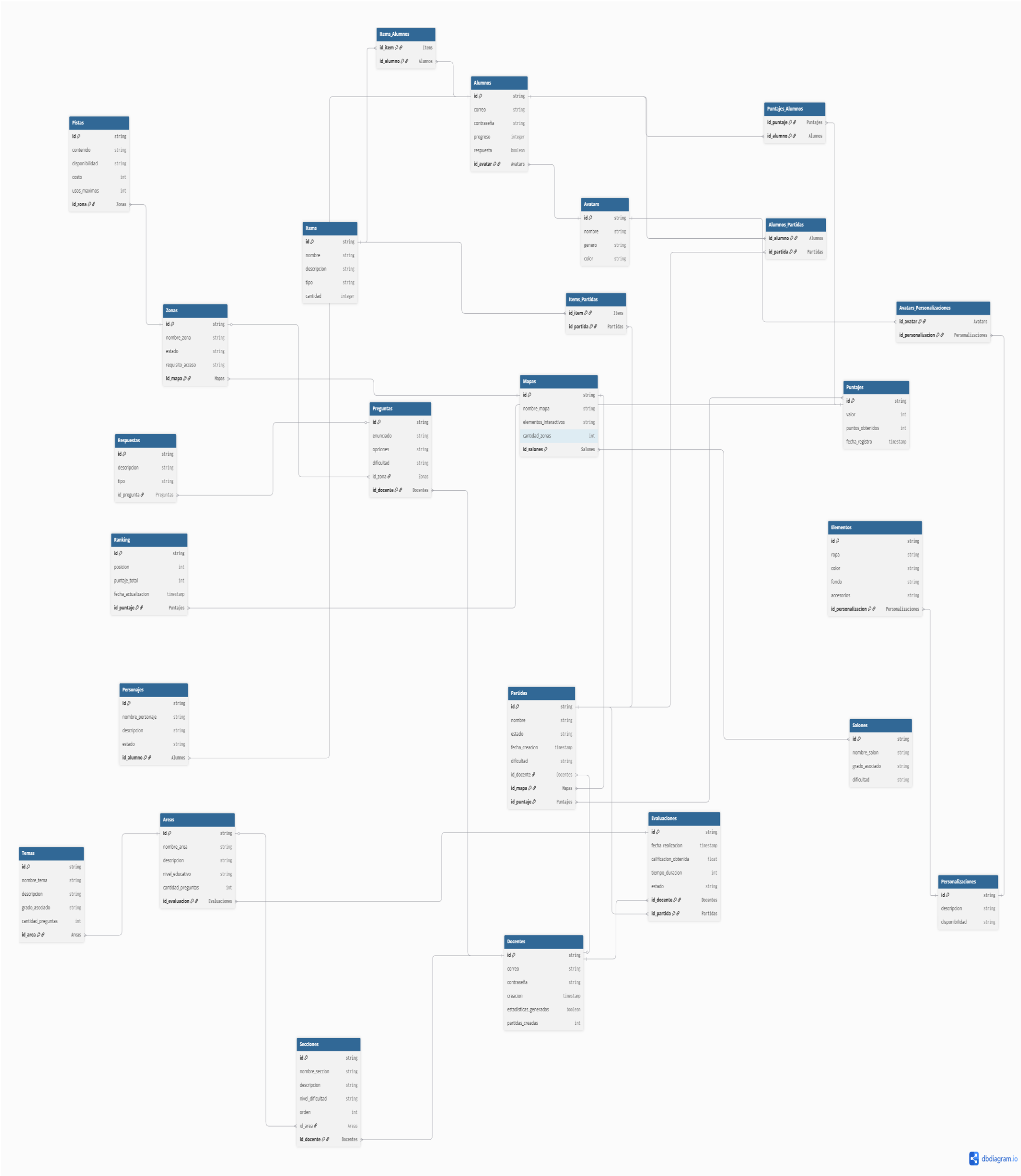
5. Modelado E/R

Diagrama de Entidad-Relación



Link: https://www.canva.com/design/DAG0It8jNCw/_d2Z459ijSrhNBLH23eN-Q/edit?utm_content=DAG0It8jNCw&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Diagrama Relacional



Link: <https://drive.google.com/file/d/1XtHPswbDGqOC5q5-JZEIGo9RA8V1o7w-/view?usp=sharing>

Script de modelo relacional

```
CREATE TABLE `Alumnos` (
  `id` string,
  `correo` string,
  `contraseña` string,
  `progreso` integer,
  `respuesta` boolean,
  `id_avatar` Avatars,
  PRIMARY KEY (`id`, `id_avatar`)
);

CREATE TABLE `Docentes` (
  `id` string PRIMARY KEY,
  `correo` string,
  `contraseña` string,
  `creacion` timestamp,
  `estadisticas_generadas` boolean,
  `partidas_creadas` int
);

CREATE TABLE `Salones` (
  `id` string PRIMARY KEY,
  `nombre_salon` string,
  `grado_asociado` string,
  `dificultad` string
);

CREATE TABLE `Partidas` (
  `id` string,
  `nombre` string,
  `estado` string,
  `fecha_creacion` timestamp,
  `dificultad` string,
  `id_docente` Docentes,
  `id_mapa` Mapas,
  `id_puntaje` Puntajes,
  PRIMARY KEY (`id`, `id_mapa`, `id_puntaje`)
);

CREATE TABLE `Secciones` (
  `id` string,
  `nombre_seccion` string,
  `descripcion` string,
  `nivel_dificultad` string,
  `orden` int,
  `id_area` Areas,
  `id_docente` Docentes,
  PRIMARY KEY (`id`, `id_docente`)
);

CREATE TABLE `Preguntas` (
  `id` string,
  `enunciado` string,
  `opciones` string,
  `dificultad` string,
  `id_zona` Zonas,
  `id_docente` Docentes,
  PRIMARY KEY (`id`, `id_docente`)
);

CREATE TABLE `Respuestas` (
  `id` string PRIMARY KEY,
  `descripcion` string,
  `tipo` string,
  `id_pregunta` Preguntas
);

CREATE TABLE `Evaluaciones` (
  `id` string,
  `fecha_realizacion` timestamp,
  `calificacion_obtenida` float,
  `tiempo_duracion` int,
  `estado` string,
  `id_docente` Docentes,
  `id_partida` Partidas,
  PRIMARY KEY (`id`, `id_docente`, `id_partida`)
);

CREATE TABLE `Areas` (
  `id` string,
  `nombre_area` string,
  `descripcion` string,
  `nivel_educativo` string,
  `cantidad_preguntas` int,
  `id_evaluacion` Evaluaciones,
  PRIMARY KEY (`id`, `id_evaluacion`)
);

CREATE TABLE `Items` (
  `id` string PRIMARY KEY,
  `nombre` string,
  `descripcion` string,
  `tipo` string,
  `cantidad` integer
);

CREATE TABLE `Personajes` (
  `id` string,
  `nombre_personaje` string,
  `descripcion` string,
  `estado` string,
  `id_alumno` Alumnos,
  PRIMARY KEY (`id`, `id_alumno`)
);

CREATE TABLE `Mapas` (
  `id` string,
  `nombre_mapa` string,
  `elementos_interactivos` string,
  `cantidad_zonas` int,
  `id_salones` Salones,
  PRIMARY KEY (`id`, `id_salones`)
);

CREATE TABLE `Personalizaciones` (
  `id` string PRIMARY KEY,
  `descripcion` string,
  `disponibilidad` string
);

CREATE TABLE `Zonas` (
  `id` string,
  `nombre_zona` string,
  `estado` string,
  `requisito_acceso` string,
  `id_mapa` Mapas,
  PRIMARY KEY (`id`, `id_mapa`)
);

CREATE TABLE `Pistas` (
  `id` string,
  `contenido` string,
  `disponibilidad` string,
  `costo` int,
  `usos_maximos` int,
  `id_zona` Zonas,
  PRIMARY KEY (`id`, `id_zona`)
);

CREATE TABLE `Ranking` (
  `id` string,
  `posicion` int,
  `puntaje_total` int,
  `fecha_actualizacion` timestamp,
  `id_puntaje` Puntajes,
  PRIMARY KEY (`id`, `id_puntaje`)
);

CREATE TABLE `Avatars` (
  `id` string PRIMARY KEY,
  `nombre` string,
  `genero` string,
  `color` string
);

CREATE TABLE `Temas` (
  `id` string,
  `nombre_tema` string,
  `descripcion` string,
  `grado_asociado` string,
  `cantidad_preguntas` int,
  `id_area` Areas,
  PRIMARY KEY (`id`, `id_area`)
);

CREATE TABLE `Puntajes` (
  `id` string PRIMARY KEY,
  `valor` int,
  `puntos_obtenidos` int,
  `fecha_registro` timestamp
);

CREATE TABLE `Elementos` (
  `id` string,
  `ropa` string,
  `color` string,
  `fondo` string,
  `accesorios` string,
  `id_personalizacion` Personalizaciones,
  PRIMARY KEY (`id`, `id_personalizacion`)
);

CREATE TABLE `Alumnos_Partidas` (
  `id_alumno` Alumnos,
  `id_partida` Partidas,
  PRIMARY KEY (`id_alumno`, `id_partida`)
);

CREATE TABLE `Items_Partidas` (
  `id_item` Items,
  `id_partida` Partidas,
  PRIMARY KEY (`id_item`, `id_partida`)
);

CREATE TABLE `Items_Alumnos` (
  `id_item` Items,
  `id_alumno` Alumnos,
  PRIMARY KEY (`id_item`, `id_alumno`)
);

CREATE TABLE `Puntajes_Alumnos` (
  `id_puntaje` Puntajes,
  `id_alumno` Alumnos,
  PRIMARY KEY (`id_puntaje`, `id_alumno`)
);

CREATE TABLE `Avatars_Personalizaciones` (
  `id_avatar` Avatars,
  `id_personalizacion` Personalizaciones,
  PRIMARY KEY (`id_avatar`, `id_personalizacion`)
);

CREATE TABLE `Puntajes_Partidas` (
  `Puntajes_id` string,
  `Partidas_id_puntaje` Puntajes,
  PRIMARY KEY (`Puntajes_id`, `Partidas_id_puntaje`)
);

CREATE TABLE `Salones_Mapas` (
  `Salones_id` string,
  `Mapas_id_salones` Salones,
  PRIMARY KEY (`Salones_id`, `Mapas_id_salones`)
);
```

Descripción de Entidades y Relaciones

Entidades:

Usuario

Representa a las personas que acceden al videojuego (alumnos, docentes, administradores).

- **Atributos:** ID (correo electrónico como identificador único), nombre, contraseña, rol, perfil.
- **Relaciones:** Un usuario puede tener varios avatares y participar en múltiples partidas. Los docentes pueden crear evaluaciones y gestionar secciones de estudiantes.

Avatar

Entidad que representa al usuario gráficamente dentro del entorno del videojuego.

- **Atributos:** ID, nombre/avatar, personalización (ropa, colores, accesorios).
- **Relaciones:** Cada avatar pertenece a un usuario y se utiliza en las partidas.

Personaje

Entidad que corresponde a rivales o figuras de la narrativa contra los cuales los alumnos se enfrentan.

- **Atributos:** ID, nombre, tipo, habilidades.
- **Relaciones:** Un personaje puede aparecer en múltiples partidas y enfrentarse a varios usuarios.

Partida

Representa cada sesión de juego en la que participa un estudiante.

- **Atributos:** ID, fecha, estado, duración.
- **Relaciones:** Una partida se desarrolla en un mapa, puede estar asociada a múltiples usuarios y genera puntajes.

Mapa

Espacio general de exploración que estructura la dinámica del juego.

- **Atributos:** ID, nombre, descripción.
- **Relaciones:** Cada mapa se compone de varias zonas y puede ser utilizado en múltiples partidas.

Zona

Subdivisión del mapa que contiene salones, actividades, ítems y pistas.

- **Atributos:** ID, nombre, tipo, estado.
- **Relaciones:** Una zona pertenece a un mapa y puede contener salones, ítems y pistas.

Salón de Aprendizaje

Lugar dentro de una zona donde se desarrollan actividades pedagógicas.

- **Atributos:** ID, nombre, descripción.
- **Relaciones:** Contiene preguntas y está asociado a temas o áreas de conocimiento.

Pregunta

Elemento central de evaluación.

- **Atributos:** ID, enunciado, nivel de dificultad, categoría.
- **Relaciones:** Una pregunta está asociada a un salón de aprendizaje y puede tener múltiples respuestas.

Respuesta

Opciones posibles para cada pregunta, de las cuales solo una es correcta.

- **Atributos:** ID, texto, valor (correcta/incorrecta).
- **Relaciones:** Cada respuesta pertenece a una única pregunta.

Ítem

Recurso estratégico recolectado en las zonas para apoyar el progreso del jugador.

- **Atributos:** ID, nombre, tipo, efecto.
- **Relaciones:** Pertenecen a una zona y pueden ser usados por los usuarios en partidas.

Pista

Elemento de ayuda que brinda información adicional al jugador.

- **Atributos:** ID, contenido, tipo (texto, audio, imagen).
- **Relaciones:** Cada pista pertenece a una zona y puede asociarse a preguntas.

Evaluación

Conjunto de preguntas organizadas por un docente.

- **Atributos:** ID, título, fecha de creación.
- **Relaciones:** Pertenecen a un área de conocimiento, puede estar asociada a diferentes secciones y asignarse a estudiantes.

Área de Conocimiento

Categoría académica principal.

- **Atributos:** ID, nombre, descripción.
- **Relaciones:** Un área contiene múltiples temas y evaluaciones.

Tema

Subdivisión dentro de un área de conocimiento.

- **Atributos:** ID, nombre, descripción.
- **Relaciones:** Un tema agrupa varias preguntas y actividades.

Puntaje

Registra los resultados de los estudiantes en una partida o evaluación.

- **Atributos:** ID, valor, fecha.
- **Relaciones:** Cada puntaje está vinculado a una partida y a un usuario.

Ranking

Muestra la clasificación de estudiantes según sus puntajes acumulados.

- **Atributos:** ID, posición, referencia a puntajes.
- **Relaciones:** Se alimenta de los puntajes individuales de los usuarios.

Relaciones:

Alumno – Partida:

Un alumno puede interactuar en múltiples partidas, y una partida puede ser interactuada por múltiples alumnos. (N.N)

Alumno – Ítem:

Un alumno puede recoger varios ítems, y un ítem puede ser recogido por varios alumnos. (N.N)

Alumno – Avatar:

Un alumno puede crear un único avatar, y un avatar es creado por un único alumno. (1.1)

Avatar – Personalización:

Un avatar puede aplicar varias personalizaciones, y una personalización puede ser aplicada por varios avatares. (N.N)

Personalización – Elemento:

Una personalización puede aplicar varios elementos, y un elemento puede ser aplicado por una única personalización. (1.N)

Alumno – Puntaje:

Un alumno puede obtener varios puntajes, y un puntaje puede ser obtenido por varios alumnos. (N.N)

Puntaje – Ranking:

Un puntaje se puede reflejar en un ranking, y un ranking puede reflejar múltiples puntajes. (1.N)

Alumno – Personaje:

Un alumno puede enfrentar a varios personajes, y un personaje puede ser enfrentado por un único alumno. (N.1)

Docente – Evaluación:

Un docente puede crear varias evaluaciones, y una evaluación es creada por un único docente. (1.N)

Docente – Partida:

Un docente puede crear varias partidas, y una partida es creada por un único docente. (1.N)

Docente – Pregunta:

Un docente puede diseñar varias preguntas, y una pregunta es diseñada por un único docente. (1.N)

Partida – Puntaje:

Una partida registra un puntaje, y un puntaje es registrado en una partida. (1.1)

Partida – Mapa:

Una partida puede usar un mapa, y un mapa puede ser usado en varias partidas. (N.1)

Partida – Ítem:

Una partida puede usar varios ítems, y un ítem puede ser usado en varias partidas. (N.N)

Mapa – Salón:

Un mapa puede simular un salón, y un salón es simulado por un mapa. (1.1)

Mapa – Zona:

Un mapa puede contener varias zonas, y una zona está contenida en un único mapa. (1.N)

Zona – Pista:

Una zona puede contener varias pistas, y una pista está contenida en una única zona. (1.N)

Zona – Pregunta:

Una zona puede ubicar varias preguntas, y una pregunta está ubicada en una única zona. (1.N)

Pregunta – Respuesta:

Una pregunta puede generar varias respuestas, y una respuesta es generada por una única pregunta. (1.N)

Reglas de Integridad Referencial

1. **Integridad Referencial entre "Preguntas" y "Temas":**
Cada registro en la colección *Preguntas* debe estar asociado con un tema existente en la colección *Temas* a través del campo *tema_id*.
2. **Integridad Referencial entre "Preguntas" y "Zonas":**
Cada registro en *Preguntas* debe estar asociado con una zona existente en la colección *Zonas* a través del campo *zona_id*.
3. **Integridad Referencial entre "Partidas" y "Docentes":**
Cada registro en la colección *Partidas* debe estar asociado con un docente existente en la colección *Docentes* mediante la clave *docente_id*.
4. **Integridad Referencial entre "Partidas" y "Estudiantes":**
Cada registro en *Partidas* debe estar asociado con un estudiante existente en la colección *Estudiantes* mediante el campo *estudiante_id*.
5. **Integridad Referencial entre "Partidas" y "Mapas":**
Cada registro en *Partidas* debe estar relacionado con un mapa existente en la colección *Mapas* mediante el campo *mapa_id*.
6. **Integridad Referencial entre "Zonas" y "Mapas":**
Cada zona registrada en la colección *Zonas* debe estar asociada a un mapa existente en *Mapas* mediante la clave *mapa_id*.
7. **Integridad Referencial entre "Items" y "Zonas":**
Cada registro en la colección *Items* debe estar asociado con una zona existente en *Zonas* mediante la clave *zona_id*.
8. **Integridad Referencial entre "Items_Alumnos" y "Items":**
Cada registro en *Items_Alumnos* debe estar asociado con un ítem existente en *Items* mediante *item_id*.
9. **Integridad Referencial entre "Items_Alumnos" y "Estudiantes":**
Cada registro en *Items_Alumnos* debe estar asociado con un estudiante existente en *Estudiantes* mediante *estudiante_id*.
10. **Integridad Referencial entre "Puntajes" y "Estudiantes":**
Cada registro en *Puntajes* debe estar asociado con un estudiante existente en *Estudiantes* mediante la clave *estudiante_id*.
11. **Integridad Referencial entre "Puntajes" y "Partidas":**
Cada puntaje registrado debe pertenecer a una partida existente en *Partidas* mediante *partida_id*.
12. **Integridad Referencial entre "Personajes" y "Estudiantes":**
Cada registro en la colección *Personajes* debe estar asociado con un estudiante existente mediante la clave *estudiante_id*.
13. **Integridad Referencial entre "Pistas" y "Zonas":**
Cada registro en la colección *Pistas* debe estar asociado con una zona existente mediante *zona_id*.
14. **Integridad Referencial entre "Evaluaciones" y "Docentes":**
Cada evaluación debe estar asociada a un docente existente mediante *docente_id*.

Colecciones (NoSLQ)

Docentes: { _id: ObjectId, correo: String, contraseña: String, estadisticas: String, partidasCreadas: Number, creacion: Date }	preguntas_id: [ObjectId] } Partidas: { _id: ObjectId, fecha: Date, estudiante_id: ObjectId, docente_id: ObjectId, mapa_id: ObjectId, puntajeFinal: Number, tiempoJugado: Number } Zonas: { _id: ObjectId, nombre: String, descripcion: String, mapa_id: ObjectId, preguntas_id: [ObjectId], items_id: [ObjectId], pistas_id: [ObjectId] } Mapas: { _id: ObjectId, nombre: String, dificultad: String, zonas_id: [ObjectId] } Items: { _id: ObjectId, nombre: String, descripcion: String,	tipo: String, zona_id: ObjectId } Items_Alumnos: { _id: ObjectId, estudiante_id: ObjectId, item_id: ObjectId, cantidad: Number } Pistas: { _id: ObjectId, texto: String, zona_id: ObjectId } Puntajes: { _id: ObjectId, estudiante_id: ObjectId, partida_id: ObjectId, puntajeObtenido: Number, fecha: Date } Personajes: { _id: ObjectId, estudiante_id: ObjectId, nombre: String, nivel: Number, experiencia: Number }
Estudiantes: { _id: ObjectId, nombre: String, correo: String, contraseña: String, progreso: Number, items_id: [ObjectId], partidas_id: [ObjectId], puntajes_id: [ObjectId], personaje_id: ObjectId }		
Preguntas: { _id: ObjectId, pregunta: String, respuestas: [String], respuestaCorrecta: String, tema_id: ObjectId, zona_id: ObjectId }		
Temas: { _id: ObjectId, nombre: String, descripcion: String,		

6. Anexos

Diagramas Adicionales

Referencias

Etapas 2: Persistencia de Datos con Backend

7. Introducción

La Etapa 2 describe el diseño, implementación y funcionamiento de la capa de persistencia de datos del sistema Dungeon School. Esta etapa detalla cómo se almacenan, consultan y administran los datos que utiliza el videojuego educativo a través de una arquitectura basada en NestJS y una base de datos NoSQL MongoDB.

La persistencia garantiza que la información crítica, como docentes, estudiantes, preguntas, zonas, ítems y partidas, permanezca disponible, consistente y segura durante todo el funcionamiento del sistema.

El backend actúa como intermediario entre el videojuego y la base de datos, suministrando una API estructurada que permite realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre cada entidad del juego. Esto permite que los usuarios interactúen con un entorno estable donde su progreso y actividades se guardan correctamente.

Propósito de la Etapa

El propósito principal de esta etapa es:

- Implementar un sistema robusto de persistencia de datos que permita almacenar de forma segura toda la información generada por el videojuego educativo.
- Diseñar y desarrollar el backend utilizando NestJS, aplicando una arquitectura modular que organice correctamente cada una de las funcionalidades del sistema.
- Definir las colecciones NoSQL, sus relaciones y sus reglas de integridad, asegurando coherencia en las operaciones.
- Establecer la comunicación fluida entre frontend y backend a través de endpoints documentados mediante Swagger
- Garantizar la escalabilidad del sistema, permitiendo que el volumen de datos crezca sin comprometer rendimiento.
- Centralizar la lógica de negocio, separando la capa visual del manejo interno de datos.

Alcance de la Etapa

La Etapa 2 abarca los siguientes elementos

- Diseño de la arquitectura del backend basada en módulos (Docentes, Estudiantes, Preguntas, Temas, Zonas, Items, Partidas, etc.).
- Modelamiento de la base de datos NoSQL, incluyendo la definición de colecciones, campos y relaciones entre documentos.
- Implementación de los controladores, servicios y schemas para manejar las entidades del sistema.
- Creación de endpoints REST para soportar todas las operaciones CRUD requeridas por el videojuego.
- Configuración de MongoDB Atlas y Mongoose para establecer la conexión y manejo de documentos.
- Documentación de la API con Swagger, permitiendo pruebas y validación de endpoints.
- Gestión de reglas de integridad referencial para garantizar consistencia de los datos.
- Pruebas básicas de operación del backend, incluyendo solicitudes POST, GET, PATCH y DELETE.

Definiciones y Acrónimos

A continuación se presentan los términos y acrónimos utilizados en esta etapa:

- **API (Application Programming Interface):**
Conjunto de reglas y rutas que permiten que el videojuego se comunique con el backend.
- **CRUD:**
Operaciones básicas de persistencia: Crear, Leer, Actualizar y Eliminar.

- **DTO (Data Transfer Object):**
Clases que definen la forma en la que los datos deben enviarse desde el cliente hacia el backend.
- **MongoDB:**
Base de datos NoSQL basada en documentos JSON extendidos.
- **Mongoose:**
Biblioteca que permite definir schemas y manejar MongoDB desde NestJS.
- **NestJS:**
Framework de Node.js para la creación de aplicaciones backend modulares y escalables.
- **Schema:**
Estructura o molde que define cómo deben lucir los documentos en una colección de MongoDB.
- **Endpoint:**
Ruta del backend que permite ejecutar una funcionalidad (ej: /docentes, /preguntas/:id).
- **Swagger:**
Herramienta para documentar y probar APIs REST de manera visual.
- **NoSQL:**
Tipo de bases de datos no relacionales, flexibles y orientadas a documentos.
- **ObjectId:**
Identificador único utilizado por MongoDB para sus documentos.

8. Diseño de la Arquitectura de Backend

Descripción de la Arquitectura Propuesta

La arquitectura del backend del sistema Dungeon School está diseñada bajo un enfoque modular, utilizando el framework NestJS, lo que permite separar cada funcionalidad del sistema en módulos independientes y altamente mantenibles.

Cada módulo contiene su propio controlador, servicio, DTOs y schema, facilitando la escalabilidad del proyecto y la reutilización de componentes.

El backend actúa como intermediario entre el frontend o el videojuego y la base de datos MongoDB. Su función es procesar solicitudes, ejecutar la lógica de negocio, validar datos, interactuar con las colecciones NoSQL y devolver respuestas consistentes al cliente.

El diseño propuesto sigue los principios de:

- Separación de responsabilidades (SoC)
- Inyección de dependencias
- Arquitectura Limpia
- Patrón Controlador–Servicio–Repositorio

Esta organización garantiza alta cohesión por módulo y bajo acoplamiento entre componentes, facilitando futuras ampliaciones del sistema (nuevas entidades, nuevas zonas, nuevos ítems, etc.).

Componentes del Backend

A continuación, se describen los elementos principales que conforman la arquitectura:

1. Controladores (Controllers)

Son los encargados de recibir las solicitudes HTTP provenientes del videojuego o de cualquier cliente que utilice la API. Los controladores exponen las rutas públicas del sistema (endpoints) y delegan toda la lógica de negocio al servicio correspondiente.

Ejemplos:

- /docentes
- /estudiantes
- /preguntas
- /partidas

2. Servicios (Services)

Contienen la lógica de negocio del sistema.

Los servicios gestionan:

- consultas a la base de datos
- validación de reglas internas
- creación, actualización y eliminación de documentos
- operaciones compuestas (consultas con múltiples colecciones)

Los servicios nunca reciben peticiones directas; solo son invocados por los controladores.

3. Schemas y Modelos (Mongoose)

Los schemas definen la estructura que deben tener los documentos almacenados en MongoDB.

Cada entidad del videojuego (como Docentes, Preguntas o Zonas) tiene su propio schema, el cual establece sus campos, tipos de datos y relaciones con otras colecciones.

Los modelos generados por Mongoose permiten realizar operaciones CRUD sobre la base de datos.

4. DTOs (Data Transfer Objects)

Aseguran que los datos enviados desde el cliente cumplan con el formato y validaciones requeridas por el sistema.

Cada endpoint posee su propio DTO, garantizando que la información ingresada sea segura y consistente.

5. Módulos (Modules)

Cada funcionalidad del sistema está organizada dentro de un módulo independiente.

Esto incluye:

- DocentesModule
- EstudiantesModule
- PreguntasModule
- ZonasModule

- TemasModule
- ItemsModule
- PartidasModule
- PersonajesModule

Los módulos agrupan controladores, servicios, schemas y DTOs relacionados.

6. Motor de Base de Datos

El backend utiliza MongoDB como base de datos NoSQL.

La conexión se realiza a través de MongooseModule, que permite mapear los schemas y crear modelos compatibles con NestJS.

7. Documentación de API (Swagger)

Swagger genera documentación visual del backend.

Permite probar los endpoints sin usar herramientas externas, visualizar los modelos de datos y validar el comportamiento de la API.

Diagramas de Arquitectura

Diagrama 1: Arquitectura General del Backend

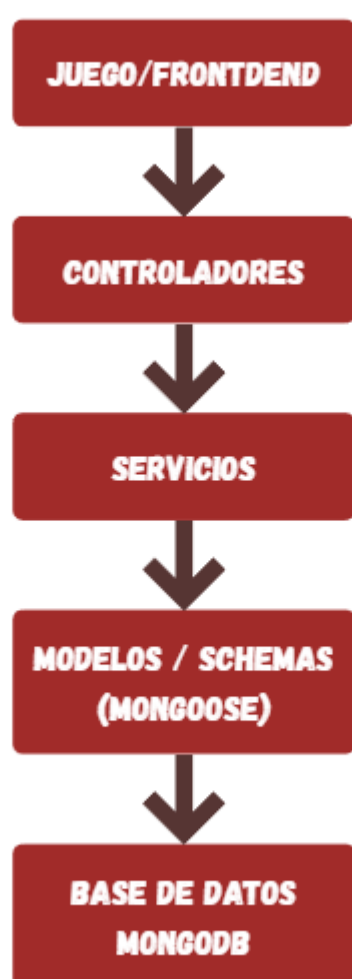


Diagrama 2: Arquitectura Modular en NestJS



Diagrama 3: Flujo de una Petición



9. Elección de la Base de Datos

Evaluación de Opciones (SQL o NoSQL)

Para la implementación del sistema Dungeon School se evaluaron dos enfoques de persistencia de datos: bases de datos relacionales (SQL) y bases de datos no relacionales (NoSQL). Cada opción presenta ventajas y limitaciones dependiendo del tipo de aplicación, la estructura de los datos y las necesidades de escalabilidad.

Bases de Datos SQL

Las bases de datos SQL (MySQL, PostgreSQL, SQL Server) utilizan un modelo estructurado basado en tablas, relaciones estrictas y esquemas rígidos.

Sus características principales son:

- Estructuras altamente definidas y estables
- Relaciones establecidas mediante claves primarias y foráneas
- Excelente para datos transaccionales y consistencia fuerte
- Menos flexibilidad para estructuras cambiantes
- Escalabilidad principalmente vertical

Bases de Datos NoSQL

Las bases de datos NoSQL (MongoDB, Firebase, Cassandra) trabajan con modelos flexibles como documentos JSON, grafos o pares clave-valor.

Sus características principales son:

- Alta flexibilidad para cambios en la estructura de datos
- Escalabilidad horizontal
- Ideal para sistemas dinámicos, distribuidos y de alto rendimiento
- Estructuras no necesariamente fijas
- Soporte para documentos complejos con anidaciones

Comparación contextualizada al proyecto

Dado que Dungeon School maneja datos dinámicos como:

- progreso del estudiante
- inventarios (ítems obtenidos)
- zonas de un mapa
- preguntas y respuestas
- partidas
- pistas
- personajes con atributos cambiantes

...es necesario un sistema flexible que permita almacenar información que no siempre tiene la misma estructura o que puede crecer con variabilidad según el contexto de juego.

Justificación de la Elección

Después de evaluar ambos enfoques, se decidió utilizar MongoDB, una base de datos NoSQL orientada a documentos, debido a los siguientes motivos:

- 1. Flexibilidad en la estructura de los datos**
El videojuego maneja documentos con campos variables, como inventarios, zonas o respuestas, lo cual encaja de manera natural con JSON.
- 2. Facilidad para modelar entidades anidadas**
MongoDB permite incluir listas, objetos y estructuras embebidas sin necesidad de tablas adicionales.
- 3. Escalabilidad horizontal**
El juego está pensado para crecer en número de estudiantes, preguntas y mapas. MongoDB soporta sharding y distribución eficiente de datos.
- 4. Rendimiento óptimo para lectura/escritura**
Operaciones como guardar partidas, actualizar puntajes o registrar ítems se realizan rápidamente gracias al modelo basado en documentos.
- 5. Integración directa con NestJS mediante Mongoose**
Mongoose facilita la definición de schemas y la manipulación de documentos sin necesidad de consultas SQL complejas.
- 6. Adecuación a datos semiestructurados**
Partidas, rutas, zonas e inventarios no requieren relaciones rígidas propias de SQL.

En conjunto, estas características hacen que MongoDB sea la opción ideal para un videojuego educativo con componentes dinámicos y un volumen de información en constante crecimiento.

Diseño de Esquema de Base de Datos

El diseño de la base de datos se realizó utilizando colecciones NoSQL que representan cada una de las entidades principales del sistema. Las colecciones fueron definidas considerando:

- independencia por módulo
- claridad en los documentos
- soporte para crecimiento progresivo
- relaciones por referencia mediante ObjectId
- simplicidad para el consumo desde el backend

A continuación, se presenta el conjunto completo de colecciones usadas en Dungeon School:

Docentes: {	creacion: Date	correo: String,
_id: ObjectId,	}	contraseña: String,
correo: String,		progreso: Number,
contraseña: String,	Estudiantes: {	items_id: [ObjectId],
estadisticas: String,	_id: ObjectId,	partidas_id: [ObjectId],
partidasCreadas: Number,	nombre: String,	puntajes_id: [ObjectId],

personaje_id: ObjectId		estudiante_id: ObjectId,
}	Zonas: {	item_id: ObjectId,
	_id: ObjectId,	cantidad: Number
Preguntas: {	nombre: String,	}
_id: ObjectId,	descripcion: String,	
pregunta: String,	mapa_id: ObjectId,	Pistas: {
respuestas: [String],	preguntas_id: [ObjectId],	_id: ObjectId,
respuestaCorrecta: String,	items_id: [ObjectId],	texto: String,
tema_id: ObjectId,	pistas_id: [ObjectId]	zona_id: ObjectId
zona_id: ObjectId	}	}
}		
	Mapas: {	Puntajes: {
Temas: {	_id: ObjectId,	_id: ObjectId,
_id: ObjectId,	nombre: String,	estudiante_id: ObjectId,
nombre: String,	dificultad: String,	partida_id: ObjectId,
descripcion: String,	zonas_id: [ObjectId]	puntajeObtenido: Number,
preguntas_id: [ObjectId]	}	fecha: Date
}		}
	Items: {	
Partidas: {	_id: ObjectId,	Personajes: {
_id: ObjectId,	nombre: String,	_id: ObjectId,
fecha: Date,	descripcion: String,	estudiante_id: ObjectId,
estudiante_id: ObjectId,	tipo: String,	nombre: String,
docente_id: ObjectId,	zona_id: ObjectId	nivel: Number,
mapa_id: ObjectId,	}	experiencia: Number
puntajeFinal: Number,		}
tiempoJugado: Number	Items_Alumnos: {	
}	_id: ObjectId,	

Estas colecciones conforman la estructura base del sistema y permiten gestionar la persistencia del videojuego de forma organizada, eficiente y escalable.

10. Implementación del Backend

Elección del Lenguaje de Programación

El backend de Dungeon School fue implementado utilizando TypeScript sobre el framework NestJS. La elección se fundamenta en los siguientes aspectos:

1. Tipado estático

TypeScript permite definir tipos estrictos, reduciendo errores y mejorando la mantenibilidad del código.

2. Estructura modular y escalable

NestJS organiza la aplicación en módulos, controladores y servicios, lo cual se adapta perfectamente a los múltiples componentes del videojuego.

3. Compatibilidad con Node.js

Permite un backend eficiente, con tiempos de respuesta adecuados para videojuegos educativos con actualizaciones frecuentes.

4. Integración nativa con MongoDB mediante Mongoose

Facilita el manejo de colecciones NoSQL y la creación de esquemas.

5. Herramientas robustas

Swagger, pipes, guards, interceptors y middlewares hacen a NestJS una plataforma completa para un sistema educativo serio.

Creación de la Lógica de Negocio

La lógica de negocio del proyecto se implementa principalmente en los services de cada módulo. Cada service contiene las operaciones necesarias para manejar las entidades del juego.

Ejemplos de lógica implementada:

- **DocentesService:** creación de docentes, gestión de estadísticas y manejo de credenciales.
- **EstudiantesService:** registro de estudiantes, progreso, inventario y puntajes.
- **PreguntasService:** creación, edición y relación de preguntas con temas y zonas.
- **PartidasService:** manejo del inicio, fin y registro de partidas.
- **ItemsService:** asignación de ítems desde las zonas a los estudiantes.
- **ZonasService:** vinculación de zonas con mapas, preguntas e ítems.
- **PersonajesService:** control de nivel, experiencia y atributos del avatar del jugador.

Cada módulo sigue el patrón:



Esto garantiza:

- Separación clara de responsabilidades
- Código reutilizable
- Mantenibilidad a largo plazo
- Facilidad para realizar pruebas

Desarrollo de Endpoints y APIs

La API del backend fue construida utilizando el sistema de controladores de NestJS. Cada controlador define rutas REST que permiten al videojuego realizar operaciones CRUD sobre las diferentes entidades.

Ejemplos de endpoints:

Docentes:	DELETE /estudiantes/:id
POST /docentes	Preguntas:
GET /docentes	POST /preguntas
GET /docentes/:id	GET /preguntas
PATCH /docentes/:id	GET /preguntas/:id
DELETE /docentes/:id	PATCH /preguntas/:id
Estudiantes:	DELETE /preguntas/:id
POST /estudiantes	Partidas:
GET /estudiantes	POST /partidas
GET /estudiantes/:id	GET /partidas
PATCH /estudiantes/:id	GET /partidas/:id

Cada ruta está documentada en Swagger, lo que permite:

- visualizar los modelos de datos
- realizar pruebas desde el navegador
- validar que las solicitudes funcionan correctamente

Autenticación y Autorización

Para garantizar un acceso seguro a la plataforma, el backend contempla un sistema de autenticación basado en JSON Web Tokens (JWT). Este mecanismo permite que solo usuarios autorizados puedan acceder a recursos protegidos.

Autenticación

Consiste en verificar la identidad de un usuario mediante:

1. Ingreso de correo y contraseña
2. Verificación de credenciales en la base de datos
3. Generación de un token JWT
4. Envío del token al cliente

El token se usa en todas las solicitudes posteriores como comprobante de identidad.

Autorización

Define qué usuarios pueden acceder a determinados recursos.

Por ejemplo:

- Solo un docente puede crear preguntas o configurar temas.
- Solo un estudiante puede acceder a su inventario o partidas.
- Usuarios no autenticados no pueden realizar ninguna acción crítica.

Esta seguridad se implementa mediante:

- Guards de NestJS
- Decoradores de roles
- Middleware de verificación de token

De esta forma, se garantiza que el sistema sea confiable, seguro y adecuado para un entorno educativo.

11. Conexión a la Base de Datos

Configuración de la Conexión

El backend de Dungeon School utiliza MongoDB Atlas como proveedor de almacenamiento en la nube, permitiendo una conexión remota, segura y escalable.

La integración se realiza mediante el módulo MongooseModule de NestJS, el cual permite definir y usar modelos de MongoDB dentro del proyecto.

La conexión se implementa en el archivo app.module.ts:

```
@Module({
  imports: [
    MongooseModule.forRoot(
      'mongodb+srv://mdilsomosquera:Sidetoside28@cluster0.nb34gqu.mongodb.net/dungeondb?appName=Cluster0'
    ),
    EstudiantesModule, DocentesModule, PreguntasModule, PersonajesModule, TemasModule,
  ],
  controllers: [AppController],
  providers: [AppService],
})
```

Cada módulo del sistema registra su schema mediante:

```
MongooseModule.forFeature([
  { name: NombreEntidad.name, schema: NombreEntidadSchema }
])
```

De este modo, el backend puede crear, leer, actualizar y eliminar documentos directamente desde la base de datos.

Desarrollo de Operaciones CRUD

El backend implementa operaciones CRUD (Create, Read, Update, Delete) en cada módulo.

Estas operaciones se desarrollan en los services, mientras que los controllers exponen las rutas a través de la API.

1. Create (Crear)

Recibe datos desde el frontend y crea un nuevo documento:

```
const nuevo = new this.modelo(dto);  
  
return nuevo.save();
```

2. Read (Leer)

Permite obtener uno o varios documentos:

```
return this.modelo.find();  
  
return this.modelo.findById(id);
```

3. Update (Actualizar)

Modifica uno o varios campos de un documento existente:

```
return this.modelo.findByIdAndUpdate(id, updateDto, { new: true });
```

4. Delete (Eliminar)

Remueve un documento de la colección:

```
return this.modelo.findByIdAndDelete(id);
```

Cada módulo (Docentes, Estudiantes, Preguntas, Zonas, Temas, Partidas, etc.) implementa estas operaciones respetando las reglas de integridad del sistema.

Manejo de Transacciones

Aunque MongoDB es una base de datos NoSQL, permite el uso de transacciones ACID cuando se requiere garantizar que múltiples operaciones se ejecuten correctamente o se reviertan por completo.

Esto es útil en situaciones como:

- Registrar una partida y al mismo tiempo actualizar el puntaje del estudiante
- Asignar un ítem y actualizar el inventario en una sola operación
- Crear una zona y asociarla directamente a un mapa

Ejemplo de manejo de transacción:

```
const session = await this.connection.startSession();  
  
session.startTransaction();
```

```
try {
```

```

await this.modelo1.create([data1], { session });

await this.modelo2.updateOne({ _id: id }, data2, { session });


await session.commitTransaction();

session.endSession();

} catch (error) {

await session.abortTransaction();

session.endSession();

throw error;

}

```

El uso de transacciones en MongoDB permite asegurar que varias operaciones relacionadas se ejecuten de manera coherente, evitando inconsistencias en los datos. Si alguna parte del proceso falla, todas las acciones realizadas dentro de la transacción se revierten automáticamente, garantizando que la base de datos permanezca íntegra y en un estado válido. En Dungeon School, este mecanismo es especialmente útil para acciones críticas que involucran múltiples colecciones, como registrar una partida mientras se actualiza el progreso del estudiante, modificar el puntaje obtenido o gestionar los cambios en el inventario del jugador. Gracias a las transacciones, estas operaciones se tratan como una sola unidad, lo que evita errores y asegura que la información del juego se mantenga confiable y consistente.

12. Pruebas del Backend

Diseño de Casos de Prueba

Para validar el correcto funcionamiento del backend de Dungeon School, se diseñaron casos de prueba orientados a evaluar las operaciones CRUD de cada módulo (Docentes, Estudiantes, Preguntas, Zonas, Temas, Ítems, Partidas y Personajes). Cada caso de prueba incluye:

- **Objetivo de la prueba** (ej. “Verificar creación exitosa de un docente”)
- **Datos de entrada** (body con DTO correcto e incorrecto)
- **Pasos a ejecutar** (realizar POST, GET, PATCH, DELETE)
- **Resultado esperado**
- **Resultado obtenido**

Los casos de prueba se diseñaron considerando escenarios:

- Con datos válidos
- Con datos incompletos
- Con formatos incorrectos
- Con IDs inexistentes
- Con peticiones duplicadas

- Con datos inconsistentes en la base

Esto permitió identificar errores en validaciones, estructura de DTOs, y respuestas de los controladores.

Ejecución de Pruebas Unitarias y de Integración

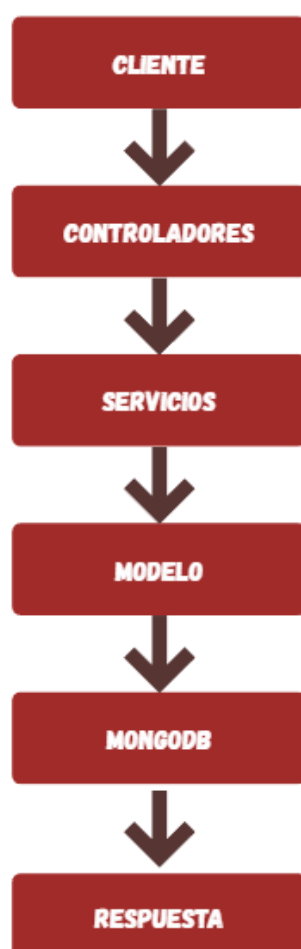
Las pruebas unitarias se enfocaron en evaluar los services, comprobando que cada método interno funcionara correctamente de manera independiente. Estas pruebas verificaron:

- búsqueda de documentos
- creación de registros
- validaciones internas
- manejo de excepciones
- actualización parcial de datos
- eliminación lógica o física

Las pruebas de integración se realizaron utilizando herramientas como:

- Swagger (incluido en NestJS)
- Postman
- Thunder Client (VSCode)

Estas pruebas permitieron evaluar el flujo completo desde:



Durante las pruebas se realizaron solicitudes reales a endpoints como:

- POST /docentes

Docentes

POST

/docentes

Parameters

No parameters

Request body

required

Edit Value

Schema

```
{
  "correo": "profe@mail.com",
  "contraseña": "123",
  "creacion": "2025-11-25",
  "estadísticas": "100",
  "partidasCreadas": 5
}
```

Curl

```
curl -X 'POST' \
'https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbh6qhra.cloudworkstations.dev/docentes' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "correo": "profe@mail.com",
  "contraseña": "123",
  "creacion": "2025-11-25",
  "estadísticas": "100",
  "partidasCreadas": 5
}'
```

Request URL

```
https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbh6qhra.cloudworkstations.dev/docentes
```

Server response

Code

Details

201

Response body

```
{
  "id": "5",
  "correo": "profe@mail.com",
  "contraseña": "123",
  "creacion": "2025-11-25",
  "estadísticas": "100",
  "partidasCreadas": 5
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-methods: GET,POST,OPTIONS,PATCH,DELETE
access-control-allow-origin: https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbh6qhra.cloudworkstations.dev
connection: keep-alive
content-length: 121
content-security-policy: frame-ancestors 'self' https://80-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbh6qhra.cloudworkstations.dev https://firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbh6qhra.cloudworkstations.dev https://monospace.corp.google.com https://monospace-dev.corp.google.com https://monospace-staging.corp.google.com https://monospace-autopush.corp.google.com https://msm.sandbox.google.com https://monospace.sandbox.google.com https://idx.sandbox.google.com https://monospace.google.com https://idx.google.com https://studio.firebase.google.com https://*.sslproxy.corp.google.com https://*.cloudworkstations.googleusercontent.com https://localhost.corp.google.com:10443
content-type: application/json; charset=utf-8
date: Wed, 03 Dec 2025 05:07:56 GMT
etag: W/"79-nH5SsJwAvGvFlq345L7Si3dQIOA"
keep-alive: timeout=5
```

- GET /estudiantes

Curl

```
curl -X 'GET' \
  'https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes' \
  -H 'accept: */*'
```

Request URL

```
https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
[{"estadisticas": 90, "partidasCreadas": 9}, {"id": "3", "correo": "Adrian@gmail.com", "contrase\u00f1a": "123", "creacion": "2023-03-03", "estadisticas": "80", "partidasCreadas": 8}, {"id": "4", "correo": "juan@gmail.com", "contrase\u00f1a": "123", "creacion": "2023-04-04", "estadisticas": "70", "partidasCreadas": 7}, {"id": "5", "correo": "profe@mail.com", "contrase\u00f1a": "123", "creacion": "2025-11-25", "estadisticas": "100", "partidasCreadas": 5}
]
```

GET

/docentes/{id}

Parameters

Name	Description
id * required string (path)	5

Execute

Responses

Curl

```
curl -X 'GET' \
'https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes/5' \
-H 'accept: */*'

```

Request URL

```
https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes/5

```

Server response

Code	Details
200	Response body <pre>{ "id": "5", "correo": "profe@mail.com", "contraseña": "123", "creacion": "2025-11-25", "estadisticas": "100", "partidasCreadas": 5 } </pre> Response headers

- PATCH /preguntas/:id

PATCH

/docentes/{id}

Parameters

Name	Description
id * required	
string (path)	5

Request body required

Edit Value | Schema

```
{"correo": "profeNuevo@mail.com"}
```

Curl

```
curl -X 'PATCH' \
'https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbhib6qhra.cloudworkstations.dev/docentes/5' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{"correo": "profeNuevo@mail.com"}'
```

Request URL

```
https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbhib6qhra.cloudworkstations.dev/docentes/5
```

Server response

Code	Details
200	Response body <pre>{ "message": "Docente actualizado", "docente": { "id": "5", "correo": "profeNuevo@mail.com", "contraseña": "123", "creacion": "2025-11-25", "estadisticas": "100", "partidasCreadas": 5 } }</pre> Response headers

<https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbb>

Server response

Code	Details
200	Response body <pre> "contraseña": "123", "creacion": "2023-02-02", "estadisticas": "90", "partidasCreadas": 9 }, { "id": "3", "correo": "Adrian@gmail.com", "contraseña": "123", "creacion": "2023-03-03", "estadisticas": "80", "partidasCreadas": 8 }, { "id": "4", "correo": "juan@gmail.com", "contraseña": "123", "creacion": "2023-04-04", "estadisticas": "70", "partidasCreadas": 7 }, { "id": "5", "correo": "profeNuevo@mail.com", "contraseña": "123", "creacion": "2025-11-25", "estadisticas": "100", "partidasCreadas": 5 }] </pre>

- DELETE /items/:id

Curl

```
curl -X 'DELETE' \
  'https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes/5' \
  -H 'accept: */*'
```

Request URL

<https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes/5>

Server response

Code	Details
200	Response body <pre> { "message": "Docente eliminado", "eliminado": [{ "id": "5", "correo": "profeNuevo@mail.com", "contraseña": "123", "creacion": "2025-11-25", "estadisticas": "100", "partidasCreadas": 5 }] } </pre>

Curl

```
curl -X 'GET' \
'https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes' \
-H 'accept: */*'

```

Request URL

```
https://3000-firebase-project-1762258421725.cluster-zhw3w37rxzgkutusbbhib6qhra.cloudworkstations.dev/docentes

```

Server response

Code	Details
200	Response body <pre> "contraseña": "123", "creacion": "2023-01-01", "estadisticas": "100", "partidasCreadas": 10 }, { "id": "2", "correo": "Mary@gmail.com", "contraseña": "123", "creacion": "2023-02-02", "estadisticas": "90", "partidasCreadas": 9 }, { "id": "3", "correo": "Adrian@gmail.com", "contraseña": "123", "creacion": "2023-03-03", "estadisticas": "80", "partidasCreadas": 8 }, { "id": "4", "correo": "juan@gmail.com", "contraseña": "123", "creacion": "2023-04-04", "estadisticas": "70", "partidasCreadas": 7 }] </pre>

Manejo de Errores y Excepciones

El backend implementa un sistema robusto de manejo de errores utilizando:

- **DTOs con validadores**
- **Excepciones personalizadas de NestJS**
- **Filtros de excepción**
- **Respuestas HTTP estandarizadas**

Esto permite detectar y responder adecuadamente a:

Errores comunes probados en Swagger:

- **400 – Bad Request**
Cuando los datos enviados no cumplen el formato esperado o faltan campos obligatorios.
- **404 – Not Found**
Cuando se intenta acceder a un recurso inexistente, por ejemplo, un docente con ID inválido.
- **409 – Conflict**
Cuando un recurso ya existe (como correos duplicados).

- **500 – Internal Server Error**

Cuando ocurre un error inesperado en la base o en la lógica del servicio.

Durante las pruebas se verificó que:

- cada error muestra un mensaje descriptivo
- las excepciones se manejan sin detener el servidor
- los controladores no exponen información interna innecesaria
- el estado HTTP coincide con el error generado

Esto asegura que los fallos puedan identificarse fácilmente y que la API mantenga estabilidad durante su ejecución.

Etapas 3: Consumo de Datos y Desarrollo Frontend

13. Introducción

Propósito de la Etapa

Alcance de la Etapa

Definiciones y Acrónimos

14. Creación de la Interfaz de Usuario (UI)

Diseño de la Interfaz de Usuario (UI) con HTML y CSS

Consideraciones de Usabilidad

Maquetación Responsiva

15. Programación Frontend con JavaScript (JS)

Desarrollo de la Lógica del Frontend

Manejo de Eventos y Comportamientos Dinámicos

Uso de Bibliotecas y Frameworks (si aplicable)

16. Consumo de Datos desde el Backend

Configuración de Conexiones al Backend

Obtención y Presentación de Datos

Actualización en Tiempo Real (si aplicable)

17. Interacción Usuario-Interfaz

Manejo de Formularios y Validación de Datos

Implementación de Funcionalidades Interactivas

Mejoras en la Experiencia del Usuario

18. Pruebas y Depuración del Frontend

Diseño de Casos de Prueba de Frontend

Pruebas de Usabilidad

Depuración de Errores y Optimización del Código

19. Implementación de la Lógica de Negocio en el Frontend

Migración de la Lógica de Negocio desde el Backend (si necesario)

Validación de Datos y Reglas de Negocio en el Frontend

20. Integración con el Backend

Verificación de la Comunicación Efectiva con el Backend

Pruebas de Integración Frontend-Backend

ANEXOS

Diagramas UML